

Learn Python

Student Edition

10 two-hour sessions + capstone · runnable examples · homework · trap cheat sheets · self-check quizzes

An accelerated, self-study Python course
for a researcher with no prior coding experience.

Offline companion to the interactive website.
Generated 2026-07-06 · Instructional content original

Learn Python — Student Syllabus

Welcome. This is a fast, ~20-hour path (**ten two-hour sessions**, plus an optional capstone) from "never coded" to "I can write a real Python program to wrangle my research data." It's re-ordered for you and front-loaded with the language quirks that trip people up.

How each 2-hour session runs: a 5-minute warm-up on last time's traps and homework, then the **core** of the topic (concept → I code, you watch → **you code**), a short break, then **Going deeper** — a second hour of genuinely new material on the same topic, with its own live examples and its own practice round — and a recap + quiz to close. You will *type every example yourself*.

Homework: every session assigns ~30–45 minutes of homework (the **Homework** section of that session's `practice.md`). It does **not** count toward class time, solutions are included, and the next session's warm-up starts from it. The **In class — going deeper** section of each practice file belongs to the session's second hour.

What you need: Python 3.11+, VS Code (or any editor), and this folder. Open the matching `slides/`, `examples/`, and `cheatsheets/` file for each session. (A full **PDF** of the course is downloadable from the website, and every session is also a **Jupyter notebook** you can run in the browser — no install.)

The 10 sessions (2 hours each, + homework)

#	Title	You'll be able to...
1	Running Python, Variables & Types	Run code (REPL + scripts), use <code>int/float/str/bool/None</code> , f-strings, casting, read a traceback
2	The Dynamic-Typing Traps	Tell <code>==</code> from <code>is</code> , predict <code>True==1 / 0.1+0.2 / 5=="5"</code> , check types with <code>isinstance</code> , truthiness
3	Control Flow: Conditionals & Loops	<code>if/elif/else</code> , chained comparisons, <code>for / while</code> , <code>break / continue</code> , <code>enumerate / zip</code> , the validation loop
4	Data Structures	<code>list/tuple/dict/set</code> , slicing, a list of dicts as a dataset, comprehensions, sorting, aliasing
5	Functions, Scope & Reusability	Write reusable functions, <code>*args/**kwargs</code> , type hints, LEGB, dodge the mutable-default bug
6	Recursion & Recursive Thinking	Base + recursive case, trace the call stack, recurse over nested data, know the limits
7	Exceptions & Defensive Code	Validate messy input with <code>try/except</code> , <code>raise</code> , EAFP, a first <code>pytest</code> test
8	Files, Libraries & Research Data	Read/write CSV survey data, <code>statistics / datetime / pathlib</code> , <code>pip install</code> , pandas teaser
9	Regular Expressions & Text Cleaning	Validate, extract, and clean real text with <code>re</code> + capture groups
10	Modules, OOP & the Pythonic Toolkit	Import modules, build a small class with <code>@property</code> , generators/ <code>map / filter / walrus</code>
11	Capstone (optional)	Build a Gradebook & Survey Analyzer end-to-end

Files for session N: `slides/session-NN-slides.md` · `examples/session-NN/`. Session 2's **type traps** are the most load-bearing material in the course — if you deeply master one session, make it that one.

Your standing toolkit (open these any time)

- `cheatsheets/traps-and-gotchas.md` — every quirk, with the wrong vs right way. *Keep this open.*
- `cheatsheets/quick-reference.md` — syntax you'll forget (slicing, f-strings, comprehensions).
- `cheatsheets/glossary.md` — plain-language definitions of every term.

How to study (specific to this material)

1. **Type, don't read.** Re-type every example in `examples/`; change one thing and predict the result *before* you run it.
2. **Predict-then-run on traps.** For Session 2 especially, guess the output, then run. The surprise *is* the lesson.

3. **Do the homework the same day.** ~30–45 min while it's fresh; check the solutions only after a real attempt.
4. **Keep a "bugs I hit" log.** When you get a traceback, write the last line + your fix. You'll build your own personalized cheat sheet (Session 2's homework starts it).
5. **Connect to what you know.** Every topic maps to research methods/stats (see `connection-map.md`). Lean on those bridges.

What "done" looks like

You finish the capstone (S11) or, at minimum, complete every session's practice + homework and score the per-session quizzes in `assessments/`. The real test: you can open a messy CSV of your own data and write a short script that summarizes it without copying from anyone.

Scope (so you're not surprised)

This makes you a confident *programmer who handles research data*. It is **not** a full data-science course — pandas, plotting, and statistical modeling get a taste, not a deep dive. Those are the natural next step once these fundamentals are solid.

Session 1

Running Python, Variables & Types

Why Python (for you)

- Free, readable, the lingua franca of research computing.
- Glue between your data, your stats, and your writing.
- Today: from zero to "I ran a program."

Mantra for the whole course: **readability wins**.

Two ways to run Python

1. **REPL** — type `python3`, get `>>>`, run one line at a time. Great for experiments.
2. **Script** — save `hello.py`, run `python3 hello.py`. Great for real work.

```
# hello.py
print("Hello, researcher")
```

Variables = labels on objects

```
n = 30          # an int
mean = 3.7      # a float
name = "Ada"    # a str
passed = True   # a bool
missing = None  # "no value"
```

`=` is an **action** ("stick this label on that object"), **not** a math equation. So `n = n + 1` is fine.

□ In stats you name quantities (`n`, `α`). Same idea — but the name can be re-pointed.

The 5 core types

Type	Example	Think
<code>int</code>	<code>30</code>	counts
<code>float</code>	<code>3.7</code>	measurements
<code>str</code>	<code>"Ada"</code>	text
<code>bool</code>	<code>True / False</code>	flags
<code>NoneType</code>	<code>None</code>	missing

`type(x)` tells you which.

Input & output

```
name = input("Your name: ")      # ⚠ ALWAYS a str
age = int(input("Your age: "))    # convert right away
print("Hi", name, "- age", age)
```

The #1 week-one trap: `input()` gives you text. `"5" + "3"` is `"53"`, not `8`.

f-strings (use these)

```
score = 87.456
print(f"{name} scored {score:.1f}")  # one decimal
print(f"{1234567:,}")               # 1,234,567
print(f"{0.873:.1%}")               # 87.3%
```

Cleaner than `+` concatenation and no type errors.

Type conversion (casting)

```
int("42")      # 42
float("3.14")   # 3.14
str(42)         # "42"
int(3.9)        # 3   (truncates, doesn't round!)
round(3.9)      # 4
```

`int(input(...))` = read text, convert to number, in one step.

Reading a traceback (don't panic)

```
Traceback (most recent call last):
  File "x.py", line 3, in <module>
    age = int(input("Age: "))
ValueError: invalid literal for int() with base 10: 'thirty'
```

Read the **LAST** line first. It names the problem: `ValueError`, and the bad value `'thirty'`.

Live demo & your turn

- Live: `greet.py`, then a "years to graduation" calculator. We'll break it once on purpose.
- You: `examples/session-01/practice.md` — build a GPA-or-BMI style script.

Going deeper

Numbers, strings & the tools around them

Arithmetic, fully

Op	Meaning	Example
<code>+ - *</code>	the usual	<code>3 * 7 → 21</code>
<code>/</code>	true division	<code>7 / 2 → 3.5</code>
<code>//</code>	floor division	<code>7 // 2 → 3</code>
<code>%</code>	remainder	<code>7 % 2 → 1</code>
<code>**</code>	power	<code>2 ** 10 → 1024</code>

Precedence follows math (`**` first, then `* / // %`, then `+ -`) — when in doubt, **parenthesize**.

Update in place: `score += 5`, `count -= 1`, `total *= 2`.

`%` earns its keep

```
n % 2 == 0          # even?
student_id % 3      # 0, 1, or 2 -> rotate into 3 discussion groups
minutes = 130
print(minutes // 60, "h", minutes % 60, "min")  # 2 h 10 min
```

□ `//` and `%` together split a quantity into "whole units + remainder" — hours/minutes, pages/sheets, groups/leftovers.

Strings have methods

```
name = " aDA loVeLace "  
name.strip()          # "aDA loVeLace"    (trim ends)  
name.strip().title()  # "Ada Lovelace"    (chain them!)  
"CS50".lower()        # "cs50"  
"a,b,c".count(",")    # 2  
"ana@uni.edu".startswith("ana") # True  
"@ " in "ana@uni.edu" # True (membership)  
len("data")           # 4
```

Methods **return a new string** — the original never changes (strings are immutable). And strings multiply: `"ab" * 3` → `"ababab"` — `print("=" * 40)` draws a divider.

f-strings, round 2 — aligned reports

```
name, score, rate = "Ana", 91.456, 0.873  
print(f"{name:<10}{score:>8.1f}{rate:>8.1%}")  
# Ana          91.5   87.3%
```

- `:<10` pad left-aligned to 10 · `:>8` right-align to 8
 - `:.1f` one decimal · `:`, thousands · `:.1%` percent
 - `{score=}` prints `score=91.456` — debugging gold.
-

`print()`, fine-tuned

```
print("Ana", "Ben", "Cara", sep=" | ") # Ana | Ben | Cara  
print("Loading", end="...")           # no newline – the next print continues  
print('She said "wow"')               # swap quote styles...  
print("She said \"wow\"")              # ...or escape with a backslash  
print("line one\nline two")            # \n = a newline inside a string
```

- `sep=` joins the arguments (default: a space); `end=` closes the line (default: `"\n"`).
 - `\n` and `\"` are **escape sequences** — one character wearing a backslash.
-

Borrowing a toolbox: `import`

```
import math
math.sqrt(144)      # 12.0
math.floor(3.9)     # 3
math.ceil(3.1)      # 4
math.pi             # 3.141592653589793
```

One line borrows a whole library. (The full import story — and `pip` — lands in Session 8.)

Ask Python itself

```
help(round)          # the manual for one function
dir(str)             # every method a string has
```

In the REPL, `_` holds the last result. These three habits replace half your web searches.

Names that don't bite

- `snake_case` for variables and functions: `class_average`, not `ClassAvg`.
- Names say **what it is**: `n_students`, not `x`.
- Constants by convention: `MAX_SCORE = 100` (Python won't enforce it — the capitals warn humans).
- Comments explain **why**, not what: the code already says what.
- Before typing, sketch the steps in plain English (**pseudocode**) — then translate.

Your turn — round 2

`examples/session-01/practice.md` → **In class — going deeper**: formatting drill, a name normalizer, and the `%`-groups exercise.

Traps recap

- `input()` → **always a string**; convert with `int()` / `float()`.
- `print(a, b)` (comma → spaces) vs `print(a + b)` (must be same type).
- `int(3.9)` truncates to `3`; use `round()` to round.

Summary

You can run code, name values, convert types, format output, and read an error. **Next**: Session 2 — the dynamic-typing traps.

Homework (before Session 2)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-01/practice.md` → Homework.

1. **Unit converter** — a script that asks for a value and reports it converted, nicely f-string formatted.
2. **Traceback drill** — break three lines on purpose; for each, read the last line and name the error.
3. **Type-prediction table** — predict `type(...)` for ten expressions, then check in the REPL.

Practice & homework

In class

Type each solution yourself. Predict output before running. Solutions at the bottom.

Task 1 — Warm-up REPL

In the REPL (`python3`), check the type of: `42` , `42.0` , `"42"` , `True` , `None` , `7/2` , `7//2` . What surprises you? (Hint: `7/2` .)

Task 2 — GPA reporter (`gpa.py`)

Ask for a name and a GPA (a decimal). Print: `"<name>'s GPA is <gpa to 2 decimals>, which is <87.5%> of a 4.0 scale."`

Task 3 — Survey age bucket (`age.py`)

Ask for an age (integer). Print the age, and the age in "months lived" ($\text{age} \times 12$). Then deliberately type `twenty` instead of a number and **read the traceback's last line**.

Task 4 — Stretch: the string trap

Without converting, what does `input("a: ") + input("b: ")` print if you type `2` then `3` ? Now fix it so it prints `5` .

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
a, b = 1, 2
a, b = b, a;          print(a, b)          # -> 2 1    (swap, no temp variable)
print(f"{a + b = }")   # -> a + b = 3    (self-documenting f-string)
print("CS50".lower(), " hi ".strip())      # -> cs50 hi
print("@" in "ana@uni.edu", len("data"))    # -> True 4    (membership + length)
```

In class — going deeper (second hour)

E1 — f-string formatting drill

Given `pi = 3.14159265`, `n = 9876543`, `rate = 0.4567`, print exactly: `3.14 · 9,876,543 · 45.7% · pi = 3.14159265` (the self-documenting form).

E2 — Weeks enrolled

Extend `age.py`: also report the age in **weeks** (52 per year), with a thousands separator.

D1 — Aligned gradebook lines

Print three students (`name`, `score`, `rate`) as aligned columns: name left-padded to 10, score right-aligned with 1 decimal, rate as a percent. All three lines must line up.

D2 — Name normalizer

Turn `" aDA lovelace "` into `"Ada Lovelace"` by *chaining* string methods, then report `len()` of the result and whether it `.startswith("Ada")`.

D3 — Whole units + remainder

1. Convert 130 minutes into `2 h 10 min` using `//` and `%`.
2. Assign student IDs `[101, 102, 103, 104, 105, 106]` to discussion groups 0/1/2 with `%`.

Homework (before Session 2)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Unit converter (`convert.py`)

Ask for a distance in miles (may be a decimal). Print kilometers (`× 1.60934`) to 2 decimals and meters with a thousands separator: `5.0 miles = 8.05 km (8,047 meters)`

H2 — Traceback drill

Break each of these on purpose in a script, run it, and copy down the LAST line of each traceback:

1. `int("ten")`
2. `"age: " + 21`
3. `print(nmae)` right after `name = "Ada"` (typo on purpose) Then, one sentence each: what does the error *name* tell you?

H3 — Type-prediction table

Predict `type(...)` (or the output) of each, then check in the REPL: `7/2` · `7//2` · `7.0//2` · `"7"*2` · `int("7")*2` · `7 == 7.0` · `None` · `input` (no parentheses!) · `print("hi")`

Solutions

In class

```
# Task 2 – gpa.py
name = input("Name: ")
gpa = float(input("GPA: "))
print(f"{name}'s GPA is {gpa:.2f}, which is {gpa/4:.1%} of a 4.0 scale.")

# Task 3 – age.py
age = int(input("Age: "))
print(f"You are {age} years old, about {age*12} months.")
# Typing "twenty" -> ValueError: invalid literal for int() with base 10: 'twenty'

# Task 4
# "2" + "3" -> "23" (string concatenation)
a = int(input("a: ")); b = int(input("b: "))
print(a + b)          # 5
```

In class — going deeper

```
pi, n, rate = 3.14159265, 9876543, 0.4567
print(f"{pi:.2f}")          # 3.14
print(f"{n:,}")             # 9,876,543
print(f"{rate:.1%}")        # 45.7%
print(f"{pi = }")           # pi = 3.14159265

# E2
age = int(input("Age: "))
print(f"That's about {age * 52:,.} weeks.")
```

```
# D1
for name, score, rate in [("Ana", 91.456, 0.873), ("Ben", 58.0, 0.412), ("Cara", 73.2, 0.65)]:
    print(f"{name:<10}{score:>8.1f}{rate:>8.1%}")

# D2
clean = " aDA lovelace ".strip().title()
print(clean, len(clean), clean.startswith("Ada"))    # Ada Lovelace 12 True

# D3
minutes = 130
print(f"{minutes // 60} h {minutes % 60} min")        # 2 h 10 min
for sid in [101, 102, 103, 104, 105, 106]:
    print(sid, "-> group", sid % 3)
```

Homework

```
# H1 – convert.py
miles = float(input("Miles: "))
km = miles * 1.60934
print(f"{miles} miles = {km:.2f} km ({round(km * 1000):,} meters)")
```

H2 — the last lines, and what the names mean:

1. `ValueError: invalid literal for int() with base 10: 'ten'` — right *type* of argument, unusable value.
2. `TypeError: can only concatenate str (not "int") to str` — the types themselves don't fit the operation.
3. `NameError: name 'nmae' is not defined` — you used a name that was never assigned (usually a typo).

```
# H3 – what type() says
7 / 2          # float – / always gives a float (3.5)
7 // 2         # int  – floor division of two ints (3)
7.0 // 2       # float – floored VALUE, float TYPE (3.0)
"7" * 2        # str  – repetition: '77', not math
int("7") * 2   # int  – 14
7 == 7.0       # bool – True (numbers compare by value)
None           # NoneType
input          # builtin_function_or_method – a function is a value too
print("hi")    # prints hi, then evaluates to None (print returns nothing)
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
score = '5'
bonus = '3'    # what input() would hand you
score + bonus
```

Result: `'53'`. These are strings (as `input()` always returns): `+` concatenates text. Convert first: `int(score) + int(bonus)`.

2.

```
age = 21
'Age: ' + age
```

Result: `TypeError: can only concatenate str (not "int") to str`. You can't glue text to a number with `+`. Use an f-string — `f"Age: {age}"` — or `print('Age:', age)`.

3.

```
gpa = 3.9
int(gpa)
```

Result: `3`. `int()` truncates toward zero, it doesn't round. Use `round(gpa)` when you mean rounding.

4.

```
name = 'ada'
name.upper()
name
```

Result: `'ada'`. Strings are immutable: `.upper()` RETURNS a new string and leaves the original alone. Assign it back: `name = name.upper()`.

Session 2

The Dynamic-Typing Traps

A name is just a label

Python won't lock a name to a type — it can point at an `int` now and a `str` later. That freedom is powerful, but it creates **traps** where the result contradicts your intuition.

This session is hands-on. Every rule below has a one-line, runnable trap in **Traps** — **predict, then run** at the bottom of the page. Predict first, then reveal.

`==` value, `is` identity

- `==` asks "same value?" — almost always what you want.
- `is` asks "same object?" — use it only for `None` (and `True` / `False`).
- `b = a` does **not** copy: both names point at one list, so mutating `a` shows through `b`. Copy with `list(a)` or `a[:]`.

☐ Same GPA = `==`. Same student = `is`.

Booleans are integers

- `bool` is a *subtype* of `int`, so `True` behaves as `1` and `False` as `0`.
- That means `5 + True` works, and `sum(flags)` counts how many are `True`.
- Identity still differs, and `isinstance(True, int)` is true while `type(True) is int` is not.

☐ Dummy coding (1/0) baked right into the language.

Numbers: int, float, division

- `/` **always** returns a float (`4 / 2` is `2.0`); `//` floors **toward** $-\infty$.
 - `3 == 3.0` compares by value, but decimals are stored in **binary**, so `0.1 + 0.2` is not exactly `0.3`.
 - Never test two floats with `==` — use `math.isclose(a, b)`. (You already never compare two measured scores for exact equality; same instinct.)
-

Comparing across types

- `==` / `!=` across types returns `False` and never crashes.
- `<` / `>` across incompatible types raises **`TypeError`** — the computer can ask "same?" but can't *rank* text against numbers.
- A list and a tuple with the same contents are **never equal**; sequences compare element by element.

Truthiness

- Falsy: `0` `0.0` `""` `[]` `{}` `set()` `None`. Truthy: everything else — including `"0"`, `"False"`, and `[0]`.
- Idiom: `if scores:` rather than `if len(scores) > 0:`.
- But convert user text first — `"0"` is truthy.

Going deeper

None, conversions & exact decimals

`None`, properly

- `None` means **absence** — "no value here" — not zero, not empty.
- Test with identity: `if x is None:` (never `== None`).
- A function without `return` hands back `None` (you'll meet this again in Session 5).
- Three different "nothings": `None` (missing) · `""` (empty text) · `0` (a real count of zero). A survey blank is `None`-shaped; a student with 0 points is not missing.

The conversion matrix

Call	Result
<code>int("42")</code>	<code>42</code>
<code>int("4.2")</code>	<code>ValueError</code>
<code>float("4.2")</code>	<code>4.2</code>
<code>int(4.9)</code>	<code>4</code> (truncates)
<code>int(float("4.2"))</code>	<code>4</code> — the two-step
<code>str(4.2)</code>	<code>"4.2"</code> (anything converts to str)

Rule: convert at the edge, once — the moment data enters, make it the right type.

nan and inf

```
bad = float("nan")      # "not a number" – a failed/missing numeric
bad == bad              # False! NaN equals nothing
import math
math.isnan(bad)         # True – the only reliable test
float("inf") > 10**100   # True – infinity beats everything
```

NaN appears in real data pipelines (pandas uses it for missing cells) — recognize it now.

Mutable vs immutable

Immutable (can't change)	Mutable (can change)
int, float, bool, str, tuple, None	list, dict, set

```
id(x)    # the object's identity – what `is` actually compares
```

Immutables are safe to share; mutables make **aliases** (`b = a` shares!). That's Session 4's headline trap — you now know *why* it happens.

When floats aren't good enough: Decimal

```
from decimal import Decimal
Decimal("0.1") + Decimal("0.2") == Decimal("0.3")    # True!
```

- Exact decimal arithmetic — build it **from strings**, not floats.
 - Use for money and any bookkeeping where `isclose` isn't acceptable.
 - Cost: wordier and slower — floats stay the everyday default.
 - (`fractions.Fraction` exists too: exact thirds, `Fraction(1, 3)`.)
-

Chained comparisons, revisited

```
1 <= likert <= 5      # both ends checked – reads like math
a == b == c           # all three equal
```

But don't chain `is` or mix operators cleverly — chain only when it reads *exactly* like the math you mean.

Your turn — round 2

`examples/session-02/practice.md` → **In class — going deeper:** the conversion-matrix prediction table, a `Decimal` re-run of `0.1 + 0.2`, and `describe(x)` telling `None` / `""` / `0` apart.

Now spring the traps

Open **Traps** — **predict, then run** below: ~18 one-line traps. For each one, **say your prediction out loud, then run it** to reveal the real result and the reason behind it.

Then in `examples/session-02/practice.md`: write `clean_score(value)` that accepts `5`, `5.0`, or `"5"` and returns a float, rejecting nonsense — safely.

Next: Session 3 — control flow: conditionals & loops.

Homework (before Session 3)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-02/practice.md` → *Homework*.

1. **Trap journal** — pick the 5 traps that surprised you most; explain each in one sentence of your own.
2. `approx_equal(a, b)` — the float comparison you'll use instead of `==`, forever.
3. `is_missing(x)` — tell `None` apart from `0`, `""`, and `False`, correctly.

Practice & homework

In class

Predict-the-output gauntlet

For each line, write *why* (one sentence). Predict, then run to check.

```
True + True           # 2      - bools are ints; True is 1
3 == 3.0              # True   - numbers compare by value
0.1 + 0.2 == 0.3      # False  - binary float rounding
5 == "5"              # False  - different types, no error
5 > "5"               # □     - can't ORDER int vs str
[1,2] == (1,2)        # False  - list vs tuple are different types
bool("0")              # True   - non-empty string is truthy
x=[1]; y=x; x.append(2); y # [1,2] - y is an alias of x
```

Build `clean_score()`

Write a function that safely turns a value into a float on a 0–100 scale:

```
def clean_score(value):
    """
    Accept 87, 87.0, or "87" and return 87.0 (a float).
    - Reject anything outside 0..100 with a clear message (return None).
    - Compare floats safely (no exact ==).
    """
```

Test it on: `87`, `87.0`, `"87"`, `"eighty"`, `120`, `True`. What does `True` do, and why? (Hint: `bool` is an `int`...)

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
x = int("257"); y = int("257")
print(x == y, x is y)           # -> True False    (equal value, different objects)
print(float("nan") == float("nan")) # -> False      (NaN equals nothing, not even
itself)
```

In class — going deeper (second hour)

E1 — isinstance drill

Predict each, then run:

```
isinstance(True, int)
type(True) is int
isinstance(3.0, int)
isinstance("3", (int, float))
isinstance(3, (int, float))
```

E2 — Same value, same object?

Without running: after `a = [1, 2]; b = [1, 2]; c = a`, what are `a == b`, `a is b`, `a is c`? Now `c.append(3)` — what is `a`? Run and check all four.

D1 — Conversion-matrix predictions

Predict each, then run: `int("42")` · `int("4.2")` · `float("4.2")` · `int(4.9)` · `int(float("4.2"))` · `str(4.2) + "!"` · `bool("False")`.

D2 — Decimal re-run

Show that `Decimal("0.1") + Decimal("0.2") == Decimal("0.3")` where floats said `False` — and check the grade weights: does `0.1 + 0.2 + 0.3 == 0.6` fix itself with `Decimal`? Why must you build Decimals from **strings**?

D3 — `describe(x)`

Return `"missing"` for `None`, `"empty"` for `""`, `"zero"` for `0 / 0.0` (but **not** `False`), else `"value"`. Prove it on `[None, "", 0, 0.0, False, "0", 5]`. (Careful: `False == 0` — Session 2's own trap. What tool tells them apart?)

Homework (before Session 3)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Trap journal

From today's ~18 traps, pick the **five** that surprised you most. For each, write down: the one-line code, your wrong expectation, and *why* Python answers differently — one sentence, your own words. (This journal is the seed of your personal cheat sheet.)

H2 — `approx_equal(a, b, tol=1e-9)`

Write the float comparison you'll use instead of `==` from now on; it must make `approx_equal(0.1 + 0.2, 0.3)` come out `True`. Do it once by hand (`abs`), once with `math.isclose`.

H3 — `is_missing(x)`

Return `True` **only** for `None` — not for `0`, `0.0`, `""`, or `False`. Prove it on all five. (Hint: this is exactly what `is` is for.)

Solutions

In class

```
import math

def clean_score(value):
    # Reject bools explicitly – they'd sneak through as ints (True == 1).
    if isinstance(value, bool):
        print(f"Rejected {value!r}: looks like a flag, not a score.")
        return None
    try:
        score = float(value)          # handles int, float, and numeric strings
    except (ValueError, TypeError):
        print(f"Rejected {value!r}: not a number.")
        return None
    if not 0 <= score <= 100:
        print(f"Rejected {value!r}: out of range 0-100.")
        return None
    return score

for v in [87, 87.0, "87", "eighty", 120, True]:
    print(v, "->", clean_score(v))
# 87->87.0, 87.0->87.0, "87"->87.0, "eighty"->None, 120->None, True->None
```

Key lesson: `float(True)` is `1.0`, so without the explicit bool check a flag would pass as a valid score. This is the `bool < int` trap in a real function.

In class — going deeper

```
# E1
isinstance(True, int)      # True – bool is a subclass of int
type(True) is int         # False – the exact type is bool
isinstance(3.0, int)      # False – float is not an int subclass
isinstance("3", (int, float)) # False – a numeric-LOOKING string is still a str
isinstance(3, (int, float)) # True – the tuple means "any of these"

# E2
a == b    # True – same values
a is b    # False – two separate list objects
a is c    # True – c is another label on a's object
# after c.append(3): a == [1, 2, 3] – mutating through c shows through a
```

```

# D1
int("42")          # 42
# int("4.2")        -> ValueError – int() parses integer text only
float("4.2")       # 4.2
int(4.9)           # 4 (truncates)
int(float("4.2"))  # 4 (the two-step)
str(4.2) + "!"     # '4.2!'
bool("False")      # True – any non-empty string is truthy!

# D2
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))          # True
print(Decimal("0.1") + Decimal("0.2") + Decimal("0.3") == Decimal("0.6")) # True
# From strings, because Decimal(0.1) would copy the float's binary error in.

# D3
def describe(x):
    if x is None:
        return "missing"
    if x == "":
        return "empty"
    if isinstance(x, bool):          # bool ⊂ int – check the flag FIRST
        return "value"
    if isinstance(x, (int, float)) and x == 0:
        return "zero"
    return "value"

for v in [None, "", 0, 0.0, False, "0", 5]:
    print(repr(v), "->", describe(v))
# None missing · "" empty · 0 zero · 0.0 zero · False value · "0" value · 5 value

```

Homework

H1 — any five, e.g.: `0.1 + 0.2 == 0.3` (binary floats), `True == 1` (bool ⊂ int), `5 == "5"` (no cross-type conversion), `[1, 2] == (1, 2)` (list ≠ tuple), `is` on equal lists (identity ≠ equality). The *why* in your own words matters more than the pick.

```

# H2
import math

def approx_equal(a, b, tol=1e-9):
    return abs(a - b) <= tol
    # or: return math.isclose(a, b, abs_tol=tol)

print(approx_equal(0.1 + 0.2, 0.3))  # True

# H3
def is_missing(x):
    return x is None                # identity – only None itself passes

for v in [None, 0, 0.0, "", False]:
    print(repr(v), "->", is_missing(v))  # only None -> True

```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
ana_scores = [88, 91]
ben_scores = [88, 91]
ana_scores is ben_scores
```

Result: `False`. `==` compares VALUES (and `ana_scores == ben_scores` is `True`); `is` compares IDENTITY — whether they are the same object in memory. Ana's and Ben's lists just happen to match. Use `==` for values and keep `is` for `None`.

2.

```
roster = ['Ana', 'Ben']
backup = roster
roster.append('Cara')
backup
```

Result: `['Ana', 'Ben', 'Cara']`. `backup = roster` does not copy: both names point at one list, so appending through `roster` shows up in `backup` too. Copy with `list(roster)` or `roster[:]`.

3.

```
attended = True
attended == 1
```

Result: `True`. `bool` is a subclass of `int`: `True == 1` and `False == 0`, which is why a spreadsheet's 1/0 lines up with Python's True/False.

4.

```
score = 5
bonus = True    # earned a bonus point?
score + bonus
```

Result: `6`. `True` acts as `1` (and `False` as `0`) in arithmetic — a quick way to add a bonus point for a flag.

5.

```
passed = [True, False, True, True]    # did each student pass?
sum(passed)
```

Result: `3`. Summing booleans counts the `True`s — a handy way to count how many students passed.

6.

```
homework = 0.1
exam = 0.2    # their weights in the final grade
homework + exam
```

Result: `0.30000000000000004`. Floats are stored in binary, where 0.1 and 0.2 have no exact representation, so the tiny errors add up.

7.

```
weight = 0.1 + 0.2    # homework + exam
weight == 0.3
```

Result: `False`. Because `0.1 + 0.2` is `0.30000000000000004`. Never test computed scores with `==`; use `math.isclose(a, b)`.

8.

```
missing_score = float('nan')
missing_score == missing_score
```

Result: `False`. `NaN` (a missing/invalid score) is defined to equal nothing, not even itself. Test it with `math.isnan(x)`.

9.

```
total = 7    # two students' scores added up
total / 2
```

Result: `3.5`. `/` is always float (true) division, so the mean comes out as `3.5`. Use `//` when you want a whole number.

10.

```
penalty = -7    # points to spread over 2 assignments
penalty // 2
```

Result: `-4`. `//` floors toward negative infinity, not toward zero, so `-7 // 2 == -4` (it deducts 4 from each, not 3).

11.

```
score = '5'    # read from a CSV
score == 5
```


Result: `False`. Python does no automatic text/number conversion, so a CSV string and a number are simply unequal (no error). Convert first: `int(score) == 5`.

12.

```
score = '5'
5 > score
```

Result: `TypeError: '>' not supported between instances of 'int' and 'str'`. Ordering a number against text raises `TypeError` — there is no sensible order. Convert first: `5 > int(score)`.

13.

```
scores_list = [88, 91]
scores_tuple = (88, 91)
scores_list == scores_tuple
```

Result: `False`. A list never equals a tuple, whatever the contents.

14.

```
passed = True
type(passed) is int
```

Result: `False`. `type()` is exact, and `passed`'s type is `bool`, not `int`. Use `isinstance` when you want subclass-aware checks.

15.

```
passed = True
isinstance(passed, int)
```

Result: `True`. `isinstance` respects subclassing, and `bool` IS a kind of `int`.

16.

```
answer = '0' # what a student typed on the survey
bool(answer)
```

Result: `True`. Any NON-EMPTY string is truthy, including '0' and 'False'. Only the empty string is falsy — so convert survey text before testing it.

17.

```
submissions = []
bool(submissions)
```

Result: `False` . Empty containers (`[]` , `{}` , `''` , `0` , `None`) are all falsy, which is why `if submissions:` reads as 'are there any?'.

18.

```
id_from_csv = int('257')
id_from_form = int('257')    # the same student ID, parsed twice
id_from_csv is id_from_form
```

Result: `False` . CPython pre-caches small ints (-5..256), so `is` accidentally looks True there; 257 built at runtime are separate objects. Compare values with `==` , never `is` .

Session 3

Control Flow: Conditionals & Loops

Part 1 — Conditionals

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"
```

Indentation defines the block. Only the **first** true branch runs.

`if` VS `elif` — not interchangeable

```
# BUG: three separate ifs – a 95 prints ALL THREE labels
if score >= 60: print("passing")
if score >= 80: print("strong")
if score >= 90: print("excellent")
```

- Stacked `if` s are **independent questions** — overlapping conditions all fire.
 - `elif` makes them **one question with branches**: first hit wins, the rest are skipped.
 - Stack `if` s only when several can *legitimately* be true at once.
-

Comparison + chained comparisons

`==` `!=` `<` `<=` `>` `>=`

```
0 <= score <= 100      # chained – reads like math
```

❑ No need for `score >= 0` and `score <= 100` — chain it.

Logical operators (and the gotcha)

```
passed and submitted    # both
late or excused          # either
not flagged              # negate
```

Short-circuit: `a and b` skips `b` if `a` is falsy. And `and / or` return an **operand, not a bool**:

```
5 and 0      # ? ← predict, then reveal in Traps below
"" or "N/A"  # the default-value idiom
```

So write `if x:` — never `if x == True`.

Part 2 — Loops

```
for s in scores:      # each element directly
    print(s)

for i in range(5):     # 0,1,2,3,4
    print(i)

while not done:       # repeat until a condition flips
    ...
```

△ `range(1, 5)` → `1,2,3,4` — **stop is excluded** (off-by-one!). `for _ in range(3):` — `_` is the convention for "just repeat; I don't use the value."

break / continue

```
for x in data:
    if x is None:
        continue      # skip this one
    if x == "STOP":
        break          # leave the loop entirely
    process(x)
```

Stop juggling indices: enumerate & zip

```
for i, name in enumerate(names):      # index + value
    print(i, name)

for name, score in zip(names, scores): # two lists together
    print(name, score)
```

□ If you write `range(len(x))`, stop — use `enumerate / zip`.

The validation loop (you'll reuse this everywhere)

```
while True:
    raw = input("Score 0-100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        score = int(raw)
        break
    print("Try again.")
```

Your turn

examples/session-03/practice.md:

1. Grade-band classifier — test the boundaries (89.999 / 90 / 90.001).
 2. Average a roster and label each student PASS/FAIL with `zip`.
 3. A robust "ask until valid" loop.
 4. The double-label bug — three stacked `if` s that should be one ladder.
-

Going deeper

More control flow

The ternary — a tiny `if` in one line

```
label = "pass" if score >= 60 else "fail"
print(f"{name}: {'✓' if attended else '✗'}")
```

For **tiny** choices only. If it needs two reads, use a real `if`.

Return the test itself

```
def is_passing(score):          # clunky
    if score >= 60:
        return True
    return False

def is_passing(score):          # Pythonic — the comparison already IS a bool
    return score >= 60
```

Then use it bare: `if is_passing(s):` — never `if is_passing(s) == True:`. ☐ Same for any yes/no helper: `is_valid`, `is_missing`, `is_full` — one `return` line each.

match/case — the readable ladder (3.10+)

```
match answer:
    case 5 | 4:
        label = "agree"
    case 3:
        label = "neutral"
    case 1 | 2:
        label = "disagree"
    case _:
        label = "invalid" # the default
```

When every branch compares the **same value** against literals, `match` beats an `elif` ladder.

for/else — search without a flag

```
for s in scores:
    if s < 60:
        print("first failing score:", s)
        break
else:
    # runs only if the loop did NOT break
    print("everyone passed")
```

The `else` belongs to the `for`. No `found = False` bookkeeping needed.

Nested loops

```
for section in ["A", "B"]:
    for student in roster:
        print(section, student)
```

- Every inner pass runs per outer pass ($2 \times 4 = 8$ lines).
- `break` exits the **inner** loop only — to escape both, put the loops in a function and `return`.

Name the pattern you're writing

Pattern	Skeleton
accumulator	<code>total = 0 ... total += x</code>
counter	<code>n = 0 ... n += 1</code>
best-so-far	<code>best = xs[0] ... if x > best: best = x</code>
sentinel	<code>while True: ... if raw == "done": break</code>

Recognizing these turns "staring at a blank editor" into "picking a skeleton".

Your turn — round 2

`examples/session-03/practice.md` → **In class — going deeper:** a `match/case` rewrite, a `for/else` search, best-so-far without `max()`, and the return-the-test rewrite.

Traps recap

- `if x == True` → just `if x:`; use `x is None` (not `== None`).
- Stacked `if`s are independent questions — they ALL fire; `elif` makes one ladder.
- `=` (assign) vs `==` (compare) — classic typo.
- `range(1, 5)` excludes 5; test your boundaries.
- Don't modify a list while looping it; prefer `enumerate / zip` over `range(len(...))`.

(More in the cheat sheet: `match / case`, the ternary, `for/else`.)

Summary

You can branch and repeat cleanly. **Next:** Session 4 — data structures.

Homework (before Session 4)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-03/practice.md` → Homework.

1. **Attendance labeler** — chained comparisons + `elif` ladder on a list of percentages.
2. **Number-guessing game** — a `while` loop with input validation and attempt counting.
3. **Leap-year checker** — one boolean expression, tested on the tricky years.

Practice & homework

In class

Task 1 — Grade-band classifier

Write `letter_grade(score)` returning A/B/C/D/F (90/80/70/60 cutoffs), "Invalid" outside 0–100.
Test the boundaries: 90, 89.999, 0, 100, -5, 101.

Task 2 — Boolean logic

1. What is `5` and `0`? `""` or `"N/A"`? Why aren't they `True / False`?
2. Rewrite `if attended == True:` the Pythonic way.

Task 3 — Average + pass/fail (loops)

Given `names = ["Ana", "Ben", "Cara", "Dev"]` and `scores = [91, 58, 73, 64]`:

1. Compute the mean with a loop and a running total.
2. Use `zip` to print `"<name>: PASS" (≥60)` or `"<name>: FAIL"`.
3. Count the passes with `sum(s >= 60 for s in scores)`.

Task 4 — Validation loop

Write a real `while True:` prompt that keeps asking until the user types an integer 0–100.

Task 5 — Trap check

Why does this skip elements, and what's the fix?

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)
```

Task 6 — The double-label bug

This is meant to print ONE label, but a 95 prints three. Explain why, then fix it **without changing any condition**:

```
if score >= 60: print("passing")
if score >= 80: print("strong")
if score >= 90: print("excellent")
```

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
nums = [80, 92, 45]
print(all(n >= 60 for n in nums))    # -> False    (45 fails)
print(any(n >= 90 for n in nums))    # -> True     (92 passes)
print(92 in nums, 60 in nums)        # -> True False
```

In class — going deeper (second hour)

E1 — Refactor to chained comparisons

Rewrite each with one chained comparison:

1. `if x >= 0 and x <= 100:`
2. `if lo < value and value < hi:`
3. `if a == b and b == c:`

E2 — Countdown

Print `10 → 1` then `Go!` — once with `range`, once with `while`. Mind the off-by-one at **both** ends.

D1 — `match/case` rewrite

Rewrite your Likert classifier with `match/case`: `5/4 → "agree"`, `3 → "neutral"`, `1/2 → "disagree"`, anything else `→ "invalid"`. Which version reads better here — and why?

D2 — `for/else` search

Find the first score below 60 in `[91, 73, 84, 58, 90]` and print it; if there is none, print `"everyone passed"` — **without** a `found` flag.

D3 — Best AND worst, one pass

Track best-so-far and worst-so-far through a single loop over `[73, 91, 58, 84]` (no `max()` / `min()`). Print both.

D4 — Return the test itself

Rewrite each as a single `return` line, then call one of them bare in an `if` (no `== True`):

```
def is_passing(score):
    if score >= 60:
        return True
    else:
        return False

def is_full_class(roster):
    if len(roster) >= 30:
        return True
    return False
```

Homework (before Session 4)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Attendance labeler

Write `label(pct) → "perfect"` (exactly 100), `"good"` (≥ 80), `"at risk"` (≥ 50), else `"critical"`; `"invalid"` outside 0–100. Loop over `[100, 92.5, 80, 79.9, 50, 12, -3, 104]` printing `pct -> label`. **Test every boundary.**

H2 — Number-guessing game

Set `secret = 37`. Loop: ask for a guess (validate it's an integer 1–100 — reuse today's validation-loop pattern), print `higher` / `lower` / `got it in N tries`. Count attempts.

H3 — Leap-year checker

`is_leap(year)` : divisible by 4, except centuries unless divisible by 400 — as **one** boolean expression.
Verify: 2024 → True, 1900 → False, 2000 → True, 2026 → False.

Solutions

In class

```
# 1
def letter_grade(score):
    if not 0 <= score <= 100: return "Invalid"
    for cutoff, g in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff: return g
    return "F"
# 90->A, 89.999->B, 0->F, 100->A, -5->Invalid, 101->Invalid

# 2
# 5 and 0 -> 0 ; "" or "N/A" -> "N/A" (and/or return an operand, not a bool)
result = "pass" if attended else "absent"      # and just `if attended:`

# 3
total = 0
for s in scores: total += s
print(total / len(scores))                    # 71.5
for name, score in zip(names, scores):
    print(f"{name}: {'PASS' if score >= 60 else 'FAIL'}")
print("passes:", sum(s >= 60 for s in scores)) # 3

# 4
while True:
    raw = input("Score 0-100: ")
    if raw.isdigit() and 0 <= int(raw) <= 100:
        print("Got", int(raw)); break
    print("Try again.")

# 5 Removing while iterating shifts indices, so elements get skipped.
xs = [x for x in xs if x % 2 != 0]            # build a new list instead -> [1, 3]

# 6 Three separate ifs are three independent questions – 95 answers yes to all.
# Chain them, most specific first, so only the FIRST hit runs:
if score >= 90:
    print("excellent")
elif score >= 80:
    print("strong")
elif score >= 60:
    print("passing")
```

In class — going deeper

```
# E1
0 <= x <= 100
lo < value < hi
a == b == c

# E2
for n in range(10, 0, -1):    # start 10, stop BEFORE 0, step -1
    print(n)
print("Go!")

n = 10
while n >= 1:
    print(n)
    n -= 1
print("Go!")
```

```
# D1
def likert_label(answer):
    match answer:
        case 5 | 4:
            return "agree"
        case 3:
            return "neutral"
        case 1 | 2:
            return "disagree"
        case _:
            return "invalid"
# match wins here: every branch compares the SAME value against literals.

# D2
for s in [91, 73, 84, 58, 90]:
    if s < 60:
        print("first failing:", s)
        break
else:
    print("everyone passed")

# D3
scores = [73, 91, 58, 84]
best = worst = scores[0]
for s in scores[1:]:
    if s > best:
        best = s
    if s < worst:
        worst = s
print("best:", best, "worst:", worst)    # 91 58
```

```
# D4
def is_passing(score):
    return score >= 60          # the comparison already IS the bool

def is_full_class(roster):
    return len(roster) >= 30

if is_passing(72):              # bare – never `== True`
    print("through!")
```

Homework

```
# H1
def label(pct):
    if not 0 <= pct <= 100:
        return "invalid"
    if pct == 100:
        return "perfect"
    if pct >= 80:
        return "good"
    if pct >= 50:
        return "at risk"
    return "critical"

for pct in [100, 92.5, 80, 79.9, 50, 12, -3, 104]:
    print(pct, "->", label(pct))
# 100 perfect · 92.5 good · 80 good · 79.9 at risk · 50 at risk · 12 critical ·
# -3 invalid · 104 invalid

# H2
secret, tries = 37, 0
while True:
    raw = input("Guess 1-100: ")
    if not (raw.isdigit() and 1 <= int(raw) <= 100):
        print("Whole number 1-100, try again.")
        continue
    tries += 1
    guess = int(raw)
    if guess < secret:
        print("higher")
    elif guess > secret:
        print("lower")
    else:
        print(f"got it in {tries} tries")
        break

# H3
def is_leap(year):
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

print(is_leap(2024), is_leap(1900), is_leap(2000), is_leap(2026))
# True False True False
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
typed_name = '' # the user left the box blank
typed_name or 'Anonymous'
```

Result: `'Anonymous'`. `or` returns the first truthy OPERAND (else the last), and `and` the first falsy — not a bool. This is the default-value idiom.

2.

```
weeks = range(1, 5)
list(weeks)
```

Result: `[1, 2, 3, 4]`. `range(start, stop)` stops BEFORE `stop`, so weeks 1–4 here.

3.

```
scores = [55, 92, 78]
all(s >= 60 for s in scores)
```

Result: `False`. `all()` is True only if EVERY item passes; 55 fails, so it's False.

Session 4

Data Structures

Four containers, four jobs

Type	Syntax	Mutable?	Use for
<code>list</code>	<code>[1, 2, 3]</code>	yes	ordered, changing collection
<code>tuple</code>	<code>(1, 2)</code>	no	fixed record / coordinates
<code>dict</code>	<code>{"k": v}</code>	yes	key → value lookup
<code>set</code>	<code>{1, 2, 3}</code>	yes	unique items

Lists & slicing

```
xs = [10, 20, 30, 40]
xs[0]      # 10      xs[-1]  # 40 (last)
xs[1:3]    # [20, 30] (stop excluded)
xs[:2]     # [10, 20]
xs[::-1]   # reversed
xs.append(50); xs.sort()      # mutate in place
```

⚠ `xs.sort()` returns **None** — it sorts in place. Use `sorted(xs)` for a new list.

Dicts = labeled records

```
student = {"name": "Ana", "gpa": 3.9}
student["name"]      # "Ana"
student.get("major", "N/A") # safe access with default
student["major"] = "Ed"      # add/update
for key, val in student.items(): ...
```

A list of dicts = a dataset ☐

```
roster = [
    {"name": "Ana", "score": 91},
    {"name": "Ben", "score": 58},
]
```

Each dict = a **row/respondent**; each key = a **variable/column**. This is your tidy dataset until pandas shows up (Session 8).

Sets: unique, fast membership

```
answers = ["yes", "no", "yes", "maybe", "no"]
set(answers)          # {'yes', 'no', 'maybe'} - dedup
"yes" in set(answers)  # True, very fast
```

Great for "distinct responses" and "have I seen this ID?"

Comprehensions

```
[s["score"] for s in roster]          # list
[s for s in roster if s["score"] >= 60] # with filter
{s["name"]: s["score"] for s in roster} # dict
{s["score"] // 10 for s in roster}     # set of score-decades
```

Read as: *expr, for each item, (optionally) if condition.*

Sorting with a key

```
sorted(roster, key=lambda s: s["score"])          # ascending
sorted(roster, key=lambda s: s["score"], reverse=True) # descending
```

```
lambda s: s["score"] = "sort by the score field."
```

TRAP: aliasing (labels, not boxes)

```
a = [1, 2, 3]
b = a          # SAME list
a.append(4)
b              # ? 🤔 ← predict (see Traps below)

b = a.copy()   # ✔ independent copy
```

`[[0]*3]*3` makes 3 references to ONE row — use `[[0]*3 for _ in range(3)]`.

Your turn

examples/session-04/practice.md:

1. Build the roster (list of dicts); sort by score.
2. `{name: score}` dict comprehension.
3. Group students into pass/fail buckets.
4. Demonstrate the aliasing trap and fix it.

Going deeper

The collections toolkit

Unpacking

```
name, score = ("Ana", 91)           # tuple unpacking
head, *rest = [10, 20, 30, 40]      # head=10, rest=[20, 30, 40]
a, b = b, a                         # the swap, revisited
for name, score in zip(names, scores): ... # unpacking IS how zip loops work
```

Dict power methods

```
student.get("major", "N/A")         # safe lookup with default
student.pop("temp", None)            # remove + return (default if absent)
groups.setdefault("pass", []).append(name) # create-if-missing, then use
d1 | d2                             # merged dict (right side wins, 3.9+)
for key, val in student.items(): ...
```

`collections.Counter` — frequencies in one line

```
from collections import Counter
counts = Counter(["yes", "no", "yes", "maybe", "yes"])
counts          # Counter({'yes': 3, 'no': 1, 'maybe': 1})
counts.most_common(1) # [('yes', 3)]
```

- Your entire "tally the survey" loop, as one expression. It IS a dict underneath.
-

`collections.defaultdict` — grouping without ceremony

```
from collections import defaultdict
by_major = defaultdict(list)          # missing key? make a [] first
for s in roster:
    by_major[s["major"]].append(s["name"])
```


Same job as `setdefault` , cleaner at scale.

Sets, the full toolkit

```
fall & spring      # intersection: both terms
fall | spring      # union: either
fall - spring      # difference: only fall
fall ^ spring      # symmetric diff: exactly one term
{1, 2} <= {1, 2, 3} # subset? True
list(dict.fromkeys(xs)) # dedupe KEEPING order (a set won't)
```

Sorting, round 2

```
sorted(roster, key=lambda s: (-s["score"], s["name"]))
# score high→low, ties broken by name A→Z
```

- Multi-key: return a **tuple** from `key` — compared element by element (Session 2!).
 - Python's sort is **stable**: equal keys keep their original order.
-

copy VS deepcopy

```
import copy
flat = a.copy()          # new outer list – inner objects still shared
full = copy.deepcopy(a)  # copies all the way down
```

Rule: nested + you'll mutate the inners → `deepcopy` . Otherwise `.copy()` is enough.

Your turn — round 2

`examples/session-04/practice.md` → **In class — going deeper:** `Counter` + `defaultdict` re-dos, a multi-key sort, and order-keeping dedupe.

Traps recap

- `=` aliases; use `.copy()` / `copy.deepcopy()` .
- `.sort()` returns `None` (in place); `sorted()` returns new.
- `list` ≠ `tuple` even with same contents (Session 2).
- `dict.get(key, default)` avoids `KeyError` .

Summary

You can store, look up, dedup, sort, and reshape data. **Next:** Session 5 — functions, scope & reusability.

Homework (before Session 5)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-04/practice.md` → Homework.

1. **Gradebook dict drill** — add, update, look up (safely), and delete students.
2. **Frequency counter** — count survey answers with a dict (no imports).
3. **Fix the grid** — reproduce the `[[0]*3]*3` shared-row bug, then build it right.

Practice & homework

In class

Start from:

```
roster = [  
    {"name": "Ana", "score": 91}, {"name": "Ben", "score": 58},  
    {"name": "Cara", "score": 73}, {"name": "Dev", "score": 64},  
]
```

Task 1 — Rank

Print names sorted by score, highest first.

Task 2 — Map (dict comprehension)

Build `{name: score}` in one line.

Task 3 — Group

Build `{"pass": [...names...], "fail": [...names...]}` using a loop.

Task 4 — Dedup

From `["A", "B", "A", "C", "B"]`, get the distinct values and how many there are.

Task 5 — Aliasing

Show that `b = roster` then `roster.append({...})` also changes `b`. Then make `b` an independent copy so it doesn't. (Hint: nested dicts → `copy.deepcopy`.)

Bonus — Pythonic idiom drill

Cover the # -> answers, predict each line, then run.

```
head, *tail = [10, 20, 30, 40]
print(head, tail)           # -> 10 [20, 30, 40]    (star-unpacking)
print({"a": 1} | {"b": 2})  # -> {'a': 1, 'b': 2}    (dict union, 3.9+)
print({1, 2, 3} & {2, 3, 4}) # -> {2, 3}        (set intersection)
print(list(zip(*[(1, 2), (3, 4)]))) # -> [(1, 3), (2, 4)] (transpose)
```

In class — going deeper (second hour)

E1 — Slicing drill

With `xs = list(range(10))`, predict: `xs[2:5]` · `xs[-3:]` · `xs[:-3]` · `xs[::2]` · `xs[::-1]` · `xs[5:2:-1]`.

E2 — Two cohorts

`fall = {"Ana", "Ben", "Cara"}`, `spring = {"Ben", "Dev"}` — who attended both terms? either term? only fall? (`&`, `|`, `-`)

D1 — counter in one line

Redo the answer tally (`["yes", "no", "yes", "maybe", "yes", "no"]`) with `collections.Counter`, and print the top answer with `.most_common(1)`.

D2 — Group with defaultdict

Group `roster` (list of dicts with `major`) into `{major: [names]}` using `collections.defaultdict(list)`.

D3 — Multi-key sort

Sort `[("Ana", 91), ("Ben", 73), ("Cara", 91)]` by score **descending**, ties by name **ascending** — one `sorted()` call with a tuple key.

D4 — Dedupe, order kept

From `["B", "A", "B", "C", "A"]` produce `["B", "A", "C"]` (first-seen order). Why does plain `set()` not guarantee this?

Homework (before Session 5)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Gradebook dict drill

Start from `gradebook = {"Ana": 91, "Ben": 58}`. Then: add Cara (73); Ben resubmits (58 → 68); look up Dev **without** a `KeyError` (default `"no record"`); delete Ben; print `name: score` lines sorted by score, highest first.

H2 — Frequency counter

Turn `answers = ["yes", "no", "yes", "maybe", "yes", "no"]` into `{"yes": 3, "no": 2, "maybe": 1}` with a plain dict and `.get(k, 0)` — no imports. Then print the most common answer with `max(counts, key=counts.get)`.

H3 — Fix the grid

Build a 3×3 grid of zeros with `[[0]*3]*3`, set `grid[0][0] = 9`, print it — see the bug with your own eyes. Rebuild it with a comprehension and prove the fix.

Solutions

In class

```
# 1
print([s["name"] for s in sorted(roster, key=lambda s: s["score"], reverse=True)])
# ['Ana', 'Cara', 'Dev', 'Ben']

# 2
name_to_score = {s["name"]: s["score"] for s in roster}

# 3
groups = {"pass": [], "fail": []}
for s in roster:
    groups["pass" if s["score"] >= 60 else "fail"].append(s["name"])

# 4
vals = ["A", "B", "A", "C", "B"]
distinct = set(vals); print(distinct, len(distinct))  # {'A', 'B', 'C'} 3

# 5
import copy
b = roster  # alias
roster.append({"name": "Eve", "score": 80})
# b now also has Eve. To stay independent:
b = copy.deepcopy(roster)  # changes to roster no longer touch b
```


Homework

```
# H1
gradebook = {"Ana": 91, "Ben": 58}
gradebook["Cara"] = 73          # add
gradebook["Ben"] = 68           # update
print(gradebook.get("Dev", "no record")) # safe lookup
del gradebook["Ben"]           # delete
for name, score in sorted(gradebook.items(), key=lambda kv: kv[1], reverse=True):
    print(f"{name}: {score}")    # Ana: 91 / Cara: 73

# H2
answers = ["yes", "no", "yes", "maybe", "yes", "no"]
counts = {}
for a in answers:
    counts[a] = counts.get(a, 0) + 1
print(counts)                   # {'yes': 3, 'no': 2, 'maybe': 1}
print(max(counts, key=counts.get)) # yes

# H3
grid = [[0] * 3] * 3
grid[0][0] = 9
print(grid)  # [[9,0,0],[9,0,0],[9,0,0]] - three labels on ONE row (aliasing!)

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid)  # [[9,0,0],[0,0,0],[0,0,0]] - independent rows
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
gradebook = [[0] * 3] * 3    # 3 students x 3 assignments
gradebook[0][0] = 9
gradebook
```

Result: `[[9, 0, 0], [9, 0, 0], [9, 0, 0]]`. `[[0]*3]*3` makes three references to ONE inner row, so editing one edits all. Build with `[[0]*3 for _ in range(3)]`.

2.

```
gpa = {'Ana': 3.9}
gpa.get('Ben')
```

Result: `None`. `.get()` returns `None` (or a default) for a missing key instead of raising — unlike `gpa['Ben']`.

3.

```
ranking = ['Ana', 'Ben', 'Cara']
ranking[::-1]
```

Result: `['Cara', 'Ben', 'Ana']`. A slice with step -1 returns a reversed COPY — a common Python idiom.

Session 5

Functions, Scope & Reusability

Defining & calling

```
def class_average(scores):  
    """Return the mean of a list of scores."""  
    return sum(scores) / len(scores)  
  
class_average([91, 58, 73])    # 74.0
```

□ A function is a formula/coding-scheme: same input → same output.

return vs print

```
def avg(xs): return sum(xs) / len(xs)    # hands value back  
def show(xs): print(sum(xs) / len(xs))  # just displays  
  
x = avg([1,2,3])    # x = 2.0  
y = show([1,2,3])   # prints 2.0, but y is None!
```

`print` shows; `return` gives the value to the next step.

Parameters: positional, keyword, default

```
def grade(score, scale=100, passing=60):  
    ...  
grade(85)                # uses defaults  
grade(85, passing=50)    # keyword arg
```

⚠ Defaults must be **immutable** (numbers, strings, `None`) — never `[]` or `{}`.

args / *kwargs

```
def total(*args):          # any number of positionals -> tuple
    return sum(args)
total(1, 2, 3)             # 6

def tag(**kwargs):         # any number of keywords -> dict
    return kwargs
tag(name="Ana", gpa=3.9) # {'name': 'Ana', 'gpa': 3.9}

func(*my_list)             # unpack list into args
func(**my_dict)            # unpack dict into kwargs
```

TRAP: mutable default argument

```
def add_student(name, roster=[]):    # ✗
    roster.append(name)
    return roster

add_student("Ana")    # ?
add_student("Ben")    # ? ← does the list persist? (Traps below)
```

The default `[]` is created **once**, at definition. Fix on next slide.

The fix: default to None

```
def add_student(name, roster=None):  # ✓
    if roster is None:
        roster = []
    roster.append(name)
    return roster
```

Rule: mutable default? Use `None` and create inside.

Scope (LEGB) & globals

Python looks up names: **Local** → **Enclosing** → **Global** → **Built-in**.

```
count = 0
def bump():
    count = count + 1    # □ UnboundLocalError
```

Assigning `count` makes it local. Avoid `global`; **return** a value and reassign instead.

Docstrings & type hints

```
def class_average(scores: list[float]) -> float:
    """Return the arithmetic mean of `scores`."""
    return sum(scores) / len(scores)
```

Type hints document intent. **They are NOT enforced at runtime** (`mypy` checks them).

Your turn

`examples/session-05/practice.md`:

1. A small grade-functions module (with docstrings + hints).
 2. Reproduce the mutable-default bug, then fix it.
 3. `summary(*scores)` using `*args`.
-

Going deeper

Functions as values

Functions are objects

```
f = letter_grade          # no () – the function itself
f(85)                     # "B"
sorted(roster, key=grade_of) # you've been passing functions since Session 4
stats = {"mean": class_average, "max": max}
stats["mean"]([91, 58, 73]) # dispatch by name
```

`key=lambda ...` was never magic — you were handing `sorted` a function.

`lambda`, honestly

```
lambda s: s["score"]      # one expression, no statements, no docstring
```

Perfect as a throwaway `key=`. The moment it needs a second line or a name to be understood — promote it to `def`.

Closures: functions that remember

```
def make_curver(bonus):
    def curve(score):
        return min(score + bonus, 100)
    return curve          # a function, with `bonus` baked in

gentle = make_curver(5)
harsh_year = make_curver(2)
gentle(96)               # 100
```

A **factory** for functions. (The lambda-in-a-loop trap earlier is this — capture happens by variable, not by value.)

A first decorator

```
def announce(f):
    def wrapper(*args, **kwargs):
        print(f"calling {f.__name__}{args}")
        return f(*args, **kwargs)
    return wrapper

@announce                # sugar for: curve = announce(curve)
def curve(score):
    return min(score + 5, 100)
```

A decorator **wraps** a function with extra behavior. You'll use them constantly (`@property`, `@cache`, `@pytest.mark...`) — now you know what the `@` does.

Locking the call signature

```
def curve(scores, *, bonus=5):    # everything after * is keyword-only
    ...
curve(xs, bonus=3)               # ✓ self-documenting
curve(xs, 3)                     # ✗ TypeError – no mystery positional flags
```

Keyword-only arguments make call sites readable a month later.

Docstrings that test themselves

```
def class_average(scores):  
    """Mean of scores.  
  
    >>> class_average([90, 80, 70])  
    80.0  
    """  
    return sum(scores) / len(scores)
```

`python -m doctest grades.py` runs every `>>>` example and complains on mismatch — documentation that can't silently rot.

Your turn — round 2

`examples/session-05/practice.md` → **In class — going deeper:** your own `apply_to_all`, a curve factory, an `@announce` decorator, and a doctest.

Traps recap

- Mutable default arg → use `None`.
- `print` ≠ `return` (forgot `return` → `None`).
- Assigning a global inside a function → `UnboundLocalError`.
- Type hints aren't enforced.

Summary

You can write reusable, documented, reproducible functions. **Next:** Session 6 — recursion.

Homework (before Session 6)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-05/practice.md` → Homework.

1. **A tiny stats library** — `validate_score`, `curve`, `summarize`, with docstrings + type hints.
2. `mean_ignoring_none(*values)` — `*args` meets data cleaning.
3. **Scope-prediction drill** — say what prints *before* you run it.

Practice & homework

In class

Task 1 — Grade-functions module

Write three functions with docstrings and type hints:

- `class_average(scores: list[float]) -> float`
- `letter_grade(score: float) -> str` (reuse Session 3)
- `pass_rate(scores: list[float], passing: float = 60) -> float` (fraction passing, 0–1)

Use bool-summing for `pass_rate` (recall `sum(s >= passing for s in scores)`).

Task 2 — Reproduce & fix the mutable-default bug

Write `add_note(text, notes=[])` that appends and returns. Call it three times and watch the list grow. Then fix it with the `None` pattern and prove each call starts fresh.

Task 3 — *args summary

Write `summary(*scores)` that returns a dict `{"n":..., "mean":..., "max":..., "min":...}`. Call it both as `summary(91, 58, 73)` and as `summary(*my_list)`.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
def f(a, *, b):          # everything after * is keyword-only
    return a, b
print(f(1, b=2))          # -> (1, 2)
print(f(**{"a": 1, "b": 9})) # -> (1, 9)  (** unpacks a dict into arguments)
```

In class — going deeper (second hour)

E1 — Keyword-only

Rewrite `letter_grade(score, plus_minus)` so that `plus_minus` **must** be passed by keyword (`letter_grade(95, plus_minus=True)`); calling it positionally should raise `TypeError`.

E2 — Docstring polish

Give `pass_rate` a docstring that states what it returns, what `passing` means, and one usage example. One short paragraph, no fluff.

D1 — Your own `map`

Write `apply_to_all(func, values)` returning a new list with `func` applied to each value. Test with `abs` and with a lambda that curves by +5.

D2 — A function factory

Write `make_curver(bonus)` returning a function that adds `bonus`, capped at 100. Build a `gentle = make_curver(5)` and a `strict = make_curver(1)` and show they differ.

D3 — A first decorator

Write `@announce` that prints `calling <name> with <args>` before running the wrapped function. Decorate your `letter_grade` and call it.

D4 — A doctest

Add a docstring with two `>>>` examples to `class_average`, then run `python -m doctest your_file.py -v` and watch them pass.

Homework (before Session 6)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — A tiny stats library

Three functions, each with a docstring and type hints:

- `validate_score(x) -> float` — accept int/float/numeric str in 0–100; `raise ValueError` otherwise (and reject `bool` — remember Session 2!),
- `curve(scores: list[float], bonus: float = 5) -> list[float]` — add the bonus, cap at 100, and **don't mutate the input list**,
- `summarize(scores: list[float]) -> dict` — `n / mean / min / max`.

H2 — `mean_ignoring_none(*values)`

`mean_ignoring_none(90, None, 80, None, 70) → 80.0`. If nothing survives the cleaning, return `None` rather than dividing by zero.

H3 — Scope prediction

What does this print — and what breaks? Predict *before* running:

```

count = 0

def tally(xs):
    total = 0
    for x in xs:
        total += x
    return total

def bump():
    count = count + 1    # ← think hard here

print(tally([1, 2, 3]))
bump()

```

Solutions

In class

```

def class_average(scores: list[float]) -> float:
    """Mean of scores."""
    return sum(scores) / len(scores)

def letter_grade(score: float) -> str:
    """A/B/C/D/F by 90/80/70/60 cutoffs."""
    for cutoff, letter in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff:
            return letter
    return "F"

def pass_rate(scores: list[float], passing: float = 60) -> float:
    """Fraction of scores >= passing (0..1)."""
    return sum(s >= passing for s in scores) / len(scores)

# Task 2
def add_note(text, notes=None):    # fixed version
    if notes is None:
        notes = []
    notes.append(text)
    return notes

# Task 3
def summary(*scores):
    return {"n": len(scores), "mean": sum(scores)/len(scores),
            "max": max(scores), "min": min(scores)}
print(summary(91, 58, 73))
print(summary(*[91, 58, 73]))

```

In class — going deeper

```
# E1
def letter_grade(score, *, plus_minus=False):  # * makes what follows keyword-only
    ...
# letter_grade(95, True)          -> TypeError
# letter_grade(95, plus_minus=True) -> ok

# E2
def pass_rate(scores, passing=60):
    """Return the fraction (0..1) of scores at or above `passing`.

    `passing` is the cutoff, 60 by default: pass_rate([70, 50, 90]) -> 0.66...
    """
```



```

# D1
def apply_to_all(func, values):
    return [func(v) for v in values]

print(apply_to_all(abs, [-3, 4, -5]))           # [3, 4, 5]
print(apply_to_all(lambda s: min(s + 5, 100), [58, 97])) # [63, 100]

# D2
def make_curver(bonus):
    def curve(score):
        return min(score + bonus, 100)
    return curve

gentle, strict = make_curver(5), make_curver(1)
print(gentle(96), strict(96))    # 100 97

# D3
def announce(f):
    def wrapper(*args, **kwargs):
        print(f"calling {f.__name__} with {args}")
        return f(*args, **kwargs)
    return wrapper

@announce
def letter_grade(score):
    for cutoff, letter in [(90, "A"), (80, "B"), (70, "C"), (60, "D")]:
        if score >= cutoff:
            return letter
    return "F"

print(letter_grade(85))           # calling letter_grade with (85,) -> B

# D4
def class_average(scores):
    """Mean of scores.

    >>> class_average([90, 80, 70])
    80.0
    >>> class_average([100])
    100.0
    """
    return sum(scores) / len(scores)

# python -m doctest your_file.py -v -> 2 passed.

```

Homework

```
# H1
def validate_score(x) -> float:
    """Return x as a float score in 0-100, or raise ValueError."""
    if isinstance(x, bool):
        raise ValueError("bool is a flag, not a score")
    score = float(x)
    if not 0 <= score <= 100:
        raise ValueError(f"{score} outside 0-100")
    return score

def curve(scores: list[float], bonus: float = 5) -> list[float]:
    """Return a NEW list with `bonus` added to each score, capped at 100."""
    return [min(s + bonus, 100) for s in scores]

def summarize(scores: list[float]) -> dict:
    """n, mean, min, max of `scores`."""
    return {"n": len(scores), "mean": sum(scores) / len(scores),
            "min": min(scores), "max": max(scores)}

# H2
def mean_ignoring_none(*values):
    clean = [v for v in values if v is not None]
    return sum(clean) / len(clean) if clean else None

print(mean_ignoring_none(90, None, 80, None, 70))    # 80.0
print(mean_ignoring_none(None, None))                # None

# H3
# tally([1, 2, 3]) prints 6 – `total` is local to tally, no conflict with anything.
# bump() raises UnboundLocalError: the assignment makes `count` local to bump,
# so `count + 1` reads a local variable that doesn't exist yet.
# The fix is not `global` – return the new value and reassign at the call site.
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
def enroll(name, roster=[]):  
    roster.append(name)  
    return roster  
enroll('Ana')  
enroll('Ben')
```

Result: `['Ana', 'Ben']`. A default value is created ONCE, at def time, so the same list persists across calls. Use `roster=None` then `roster = roster or []`.

2.

```
def show_score(s):  
    print(s)  
show_score(91) is None
```

Result: `True`. Printing is not returning. A function with no `return` gives `None`.

3.

```
makers = [lambda: score for score in [88, 91, 73]]  
[make() for make in makers]
```

Result: `[73, 73, 73]`. Closures capture the VARIABLE `score`, not its value; by call time the loop has left `score = 73`. Fix by binding it: `lambda score=score: score`.

Session 6

Recursion & Recursive Thinking

The shape of every recursion

Real example: how many prerequisites deep is a course?

```
prereq_of = {"ED700": "ED600", "ED600": "ED500",
            "ED500": "ED400", "ED400": None}    # ED400 has none

def prereqs_deep(course):
    earlier = prereq_of[course]
    if earlier is None:                        # BASE CASE — stop here
        return 0
    return 1 + prereqs_deep(earlier)         # RECURSIVE CASE — step back one
```

Two parts, always:

- a **base case** that stops, and
 - a **recursive case** that moves *toward* the base case.
-

Trace the call stack

`orderings(n)` = ways to rank n students = $n!$

```
orderings(3)
= 3 * orderings(2)
= 3 * (2 * orderings(1))
= 3 * (2 * 1)           # base case returns 1
= 6
```

Each call waits on the one inside it. The calls stack up, then unwind.

□ Each pending call is a **stack frame** — that matters in a moment.

Recursion vs iteration

```
def orderings(n):                # ways to rank n students (n!)
    if n <= 1:
        return 1
    return n * orderings(n - 1)  # ← must RETURN the call

def orderings_loop(n):
    total = 1
    for k in range(2, n + 1):
        total *= k
    return total
```

Same answer. For flat counting, the **loop** is usually clearer.

Where recursion shines: nested data

```
def deep_sum(obj):
    if isinstance(obj, (int, float)):
        return obj
    if isinstance(obj, dict):
        return sum(deep_sum(v) for v in obj.values())
    if isinstance(obj, (list, tuple)):
        return sum(deep_sum(x) for x in obj)
    return 0

deep_sum([1, [2, [3, 4]], {"a": 6}])  # 16
```

Nested JSON, folder trees, threaded replies — a single loop can't reach all the way down. Recursion can.

The trap: no base case

```
def runaway(n):
    return runaway(n + 1)    # never stops
```

```
RecursionError: maximum recursion depth exceeded
```

Python has **no tail-call optimization** — every call keeps its frame (default limit ≈ 1000). Deep recursion *will* hit the ceiling.

Your turn

examples/session-06/practice.md:

1. Recursive `total(scores)` — sum a list of scores; name the base case first.
 2. `flatten([1, [2, [3, 4]], 5])` → one flat list.
 3. `depth(...)` — how deeply is a list nested?
-

Going deeper

Recursion in the wild

Memoization: remember past calls

```
def fib(n):                                # naive: fib(35) recomputes fib(5) thousands of times
    return n if n < 2 else fib(n - 1) + fib(n - 2)

import functools
@functools.cache                           # remembers every (input -> output) it has seen
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Naive `fib(35)`: seconds. Cached: instant. Same code — a decorator (Session 5!) did the work. When a recursion re-solves the **same subproblem**, cache it.

Divide & conquer: binary search

```
def find(sorted_names, target, lo=0, hi=None):
    if hi is None:
        hi = len(sorted_names)
    if lo >= hi:
        return False                # base case: empty range
    mid = (lo + hi) // 2
    if sorted_names[mid] == target:
        return True
    if sorted_names[mid] < target:
        return find(sorted_names, target, mid + 1, hi)
    return find(sorted_names, target, lo, mid)
```

Each call **halves** the problem: 1 000 names ≈ 10 steps, a million ≈ 20. (Requires sorted data — Session 4's `sorted` earns its keep.)

Tree-shaped data

```
org = {"name": "Dean",
      "reports": [{"name": "Chair A", "reports": []},
                  {"name": "Chair B",
                   "reports": [{"name": "Prof C", "reports": []}]}]}

def count_people(node):
    return 1 + sum(count_people(r) for r in node["reports"])

count_people(org)      # 4
```

Org charts, folder trees, threaded replies, nested codes — the function's shape **mirrors the data's shape**. That's the whole trick.

Escaping the limit: an explicit stack

```
def flatten_iter(xs):
    out, to_visit = [], list(xs)
    while to_visit:
        x = to_visit.pop(0)
        if isinstance(x, list):
            to_visit = x + to_visit    # open the box in place
        else:
            out.append(x)
    return out
```

Same logic, **no call-stack frames** — depth 10 000 is fine. When data may nest deeper than ~1000, carry your own to-visit list.

Choosing: loop, recursion, or cache?

Situation	Reach for
flat sequence	a plain loop
self-similar / nested data	recursion
repeated subproblems	recursion + @cache
possibly very deep	loop + explicit stack

Recursion is a tool, not a virtue — pick by the *shape of the data*.

Your turn — round 2

`examples/session-06/practice.md` → **In class — going deeper:** binary search with a step counter, an org-tree count, and `flatten` without recursion.

Traps recap

- Every recursion needs a **reachable base case**, or it overflows the stack.
- **Return** the recursive call — forgetting to gives you a silent `None`.
- Recursion isn't free: each call costs a stack frame (no tail-call optimization).
- A plain **loop** is better for flat sequences and for very deep work.

Summary

You can solve problems that are defined in terms of themselves — especially nested data. **Next:** Session 7 — exceptions & defensive code.

Homework (before Session 7)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-06/practice.md` → Homework.

1. `sum_digits(n)` — recursion on a number instead of a list.
2. `deep_count(obj)` — count every score in a nested gradebook dict.
3. **In your own words** — one paragraph: when is a loop the better tool?

Practice & homework

In class

Task 1 — Recursive total

Write `total(scores)` that sums a list of scores **with recursion** (no loop): `scores[0] + total(rest)`, with the empty list as the base case. Name the base case out loud first. Test `total([91, 58, 73])` and `total([])`.

Task 2 — Recursion vs iteration

Write `reverse(s)` that reverses a string recursively. Then write the loop version. Which reads more clearly to you? Test `reverse("data")`.

Task 3 — Flatten nested data

Write `flatten(xs)` that turns a list-of-lists (nested to any depth) into one flat list: `flatten([1, [2, [3, 4]], 5]) → [1, 2, 3, 4, 5]`. This is the move for nested JSON/exports.

Task 4 — How deep does it go?

Write `depth(xs)` returning how deeply a list is nested: `depth([1, [2, [3, [4]]]]) → 4`, `depth([1, 2, 3]) → 1`, `depth(5) → 0`.

Task 5 — Trap check

1. Why does this raise `RecursionError`, and what's the fix? `python def f(n): return n + f(n - 1)`
2. This returns `None` instead of a number — why? `python def orderings(n): if n <= 1: return 1
n * orderings(n - 1)`
3. Name one case where a plain loop is the better choice over recursion.

Bonus — Pythonic idiom drill

One decorator makes exponential recursion instant by remembering past calls.

```
import functools

@functools.cache          # memoize: each n is computed once
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
print(fib(35))            # -> 9227465    (try this WITHOUT @cache... then wait)
```

In class — going deeper (second hour)

E1 — Recursive countdown

`countdown(3)` prints `3 2 1 Go!`. Say the base case out loud before you type.

E2 — Trace on paper

Write the full expansion of `total([5, 10, 20])` (Task 1) the way the slides traced `orderings(3)` — every pending call, then the unwind.

D1 — Binary search, counting steps

Write recursive `find(sorted_names, target)` that also counts how many names it looked at. Search a sorted list of 7 names, then imagine 1,000 — how many steps, roughly, and why?

D2 — Count the org tree

With `org = {"name": ..., "reports": [...]}` nested to any depth, write `count_people(node)`. Then `deepest(node)` returning how many levels the tree has.

D3 — Flatten without recursion

Rewrite `flatten` using an explicit to-visit list (no recursive calls). Verify it matches your recursive version on `[1, [2, [3, 4]], 5]` — and explain why this one survives depth 10,000.

Homework (before Session 7)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — `sum_digits(n)`

`sum_digits(4823) → 17`, recursively. (Hint: `n % 10` is the last digit, `n // 10` is the rest. What's the smallest number whose digit-sum is obvious?)

H2 — `deep_count(obj)`

Count how many **numbers** (int/float — but not bool!) appear anywhere in a nested structure. Handle dicts, lists/tuples, and scalars. `deep_count({"quiz": [90, 85], "final": {"written": 88, "oral": 92}, "note": "great"}) → 4`.

H3 — In your own words

One paragraph: for which shapes of problem is a plain loop the better tool, and what *specifically* goes wrong with deep recursion around depth ~1000? (Name the error.)

Solutions

In class

```
# 1
def total(scores):
    if not scores:
        return 0
    return scores[0] + total(scores[1:])
print(total([91, 58, 73]), total([]))

# 2
def reverse(s):
    if s == "":
        return ""
    return reverse(s[1:]) + s[0]
print(reverse("data"))
# loop version: "".join(reversed(s)) - usually clearer for flat strings

# 3
def flatten(xs):
    out = []
    for x in xs:
        if isinstance(x, list):
            out.extend(flatten(x))
        else:
            out.append(x)
    return out
print(flatten([1, [2, [3, 4]], 5]))

# 4
def depth(xs):
    if not isinstance(xs, list):
        return 0
    return 1 + max((depth(x) for x in xs), default=0)
print(depth([1, [2, [3, [4]]]]), depth([1, 2, 3]), depth(5))

# 5
# 1) No reachable base case -> the calls never stop -> stack overflows.
# Fix: add `if n == 0: return 0` (or n <= 0) at the top.
# 2) The recursive case computes n*fact(n-1) but never RETURNS it,
# so the function falls off the end and returns None. Add `return`.
# 3) A loop is better when the work is a simple flat sequence, or when the
# depth could exceed ~1000 (Python has no tail-call optimization, so deep
# recursion hits RecursionError where a loop would be fine).
```

In class — going deeper

```
# E1
def countdown(n):
    if n == 0:          # base case
        print("Go!")
        return
    print(n)
    countdown(n - 1)    # recursive case, moving toward 0

# E2 – the trace
# total([5, 10, 20])
# = 5 + total([10, 20])
# =    10 + total([20])
# =        20 + total([])
# =            0          <- base case
# unwind: 20 + 0 = 20 ; 10 + 20 = 30 ; 5 + 30 = 35
```

```

# D1
def find(names, target, lo=0, hi=None, steps=1):
    if hi is None:
        hi = len(names)
    if lo >= hi:
        return False, steps
    mid = (lo + hi) // 2
    if names[mid] == target:
        return True, steps
    if names[mid] < target:
        return find(names, target, mid + 1, hi, steps + 1)
    return find(names, target, lo, mid, steps + 1)

roster = sorted(["Ana", "Ben", "Cara", "Dev", "Eve", "Fay", "Gus"])
print(find(roster, "Fay"))      # (True, 2-3 steps)
# 1,000 names -> ~10 steps: each step halves the range (2**10 = 1024).

# D2
def count_people(node):
    return 1 + sum(count_people(r) for r in node["reports"])

def deepest(node):
    if not node["reports"]:
        return 1
    return 1 + max(deepest(r) for r in node["reports"])

# D3
def flatten_iter(xs):
    out, to_visit = [], list(xs)
    while to_visit:
        x = to_visit.pop(0)
        if isinstance(x, list):
            to_visit = x + to_visit
        else:
            out.append(x)
    return out

print(flatten_iter([1, [2, [3, 4]], 5]))  # [1, 2, 3, 4, 5]
# No recursive calls -> no stack frames -> the ~1000-frame limit never applies.

```

Homework

```
# H1
def sum_digits(n):
    if n < 10:                # single digit: the sum is itself
        return n
    return n % 10 + sum_digits(n // 10)

print(sum_digits(4823))      # 17

# H2
def deep_count(obj):
    if isinstance(obj, bool):    # bool < int — a flag isn't a number (Session 2!)
        return 0
    if isinstance(obj, (int, float)):
        return 1
    if isinstance(obj, dict):
        return sum(deep_count(v) for v in obj.values())
    if isinstance(obj, (list, tuple)):
        return sum(deep_count(x) for x in obj)
    return 0

print(deep_count({"quiz": [90, 85], "final": {"written": 88, "oral": 92},
                  "note": "great"}))  # 4
```

H3 — model answer: flat sequences (summing a list, walking rows) read best as loops. Every pending recursive call keeps a stack frame, and Python has no tail-call optimization, so around ~1000 frames it raises `RecursionError` — where a loop would simply keep going. Recursion earns its keep when the *data itself* is nested/self-similar.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
def grade(n):  
    return grade(n - 1)    # forgot the base case  
grade(3)
```

Result: `RecursionError: maximum recursion depth exceeded`. Every recursive function needs a base case. Without one Python stops at its recursion limit (~1000 deep) with `RecursionError`.

2.

```
def curve(score):  
    if score >= 95:  
        return 100  
    score + 5    # looks right...  
curve(80) is None
```

Result: `True`. The second branch computes `score + 5` and then throws it away: without `return` in front, the function falls off the end and hands back `None`. In a recursive function this same slip silently poisons the whole call chain.

3.

```
import sys  
sys.getrecursionlimit()
```

Result: `1000`. Every pending call keeps a stack frame, and CPython caps the stack (~1000 by default), raising `RecursionError` past it. Recursion isn't free — deep flat work belongs in a loop.

Session 7

Exceptions & Defensive Code

try / except

```
try:
    n = int(value)
except ValueError:
    n = None          # handle the bad case
```

When code might fail at runtime, wrap it. The `except` catches the named error. Keep the `try` block **small** — only the line that can fail — so a catch can't accidentally cover code you never meant to forgive.

Common exception types

Exception	Happens when
<code>ValueError</code>	right type, bad value: <code>int("N/A")</code>
<code>TypeError</code>	wrong type: <code>5 > "5"</code>
<code>KeyError</code>	missing dict key: <code>d["nope"]</code>
<code>IndexError</code>	bad list index: <code>xs[99]</code>
<code>ZeroDivisionError</code>	<code>x / 0</code>
<code>FileNotFoundError</code>	<code>open("missing.csv")</code>

try / except / else / finally

```
try:
    n = int(value)
except ValueError:
    print("not a number")
else:
    print("ok:", n)          # only if NO exception
finally:
    print("always runs")    # cleanup
```

Raise your own

```
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError(f"{n} not in 1-5")
    return n
```

`raise` throws an exception on purpose — caller decides how to handle it.

EAFP vs LBYL

```
# LBYL – "look before you leap"
if value.isdigit():
    n = int(value)

# EAFP – "easier to ask forgiveness" (Pythonic)
try:
    n = int(value)
except ValueError:
    n = None
```

Both valid. EAFP shines when "checking first" is hard or racy.

assert (developer check, not validation)

```
assert len(scores) > 0, "scores must not be empty"
```

For *your* sanity checks while developing. Can be disabled (`python -O`), so **never** use `assert` to validate untrusted input — use `raise`.

A first test with pytest

```
# clean.py
def clean_likert(n):
    if not 1 <= n <= 5:
        raise ValueError("1-5 only")
    return n

# test_clean.py
import pytest
from clean import clean_likert

def test_valid():
    assert clean_likert(3) == 3
def test_invalid():
    with pytest.raises(ValueError):
        clean_likert(9)
```

Run: `pytest`

Your turn

`examples/session-07/practice.md`:

1. `safe_int(value)` returning `int` or `None`.
 2. Clean a dirty survey list, collecting good values + a rejection log.
 3. Write one `pytest` test.
-

Going deeper

Robust programs

The exception family tree

```
Exception
├── ValueError          ├── KeyError / IndexError (LookupError)
├── TypeError          └── OSError (FileNotFoundError, ...)
```

```
try:
    n = int(raw)
except (ValueError, TypeError):  # catch a tuple of specifics
    ...
```

Order matters: `except` clauses are tried top-down — specific before broad, and a final `except Exception as e:` only to log-and-stop, never to ignore.

Custom exceptions that carry data

```
class SurveyError(ValueError):
    def __init__(self, value, message):
        super().__init__(message)
        self.value = value          # keep the evidence

raise SurveyError(raw, f"{raw!r} is not a 1-5 rating")
```

Callers can catch `SurveyError` specifically — or `ValueError` broadly — and still see *which* value broke.

`raise ... from` — keep the cause

```
try:
    n = int(cell)
except ValueError as e:
    raise SurveyError(cell, "bad rating cell") from e
```

The traceback shows **both**: your domain-level error on top, the real low-level cause beneath. Future-you debugging at midnight says thanks.

`finally` and the cleanup mindset

```
try:
    f = open("data.csv")
    ...
finally:
    f.close()          # runs no matter what happened above
```

`with open(...)` (Session 8) is exactly this pattern, packaged. Rule: whoever acquires a resource guarantees its release.

`logging` beats `print` for diagnostics

```
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
logging.info("processing %s rows", len(rows))
logging.warning("rejected %r: out of range", raw)
```

- `print` is for the program's *output*; `logging` is for its *diary*.
 - Levels (`DEBUG/INFO/WARNING/ERROR`) let you silence chatter without deleting lines.
-

pytest, round 2

```
import pytest
from clean import clean_likert

@pytest.mark.parametrize("bad", ["3", True, None, 0, 9])
def test_rejects(bad):
    with pytest.raises(ValueError):
        clean_likert(bad)
```

One test, five inputs, five reported results. Pattern: **arrange, act, assert** — and test the *edges* (boundaries, wrong types), not the happy middle.

A function that only `print`s can't be asserted. **Return** the value, print at the edge — that's what makes code testable (Session 5's `return` vs `print`, cashing in).

Your turn — round 2

`examples/session-07/practice.md` → **In class — going deeper:** fix an except-order bug, build `SurveyError` with `raise ... from`, switch the rejection log to `logging`, and parametrize your tests.

Traps recap

- **Never** bare `except:` — name the exception.
- Don't catch too broadly or swallow errors silently.
- `assert` ≠ input validation (use `raise`).
- Catch the *specific* error you expect.

Summary

You can validate messy input and fail loudly when you should. **Next:** Session 8 — files & research data.

Homework (before Session 8)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-07/practice.md` → Homework.

1. `ask_int(prompt, lo, hi)` — Session 3's validation loop rebuilt the EAFP way.
2. **Three more pytest cases** — edge cases for `clean_likert` (`True`, `"3"`, `None`).
3. **Error triage** — five real error messages: name the exception and the fix.

Practice & homework

In class

Task 1 — `safe_int`

Write `safe_int(value)` returning `int(value)` or `None` on failure. Test on `"42"`, `"N/A"`, `""`, `None`, `3.0`.

Task 2 — Clean a survey column

Given `raw = ["5", "3", "N/A", "7", "", "1", "two", "4"]`, produce:

- `clean` — list of valid Likert ints (1–5), and

- `rejected` — list of `(value, reason)` pairs. Use a `clean_likert(n)` that **raises** `ValueError` for out-of-range or non-ints.

Task 3 — Write a test

Put `clean_likert` in `clean.py` and write `test_clean.py` with `pytest`: one passing case and one `pytest.raises(ValueError)` case. Run `pytest`.

Task 4 — Discuss

Why is `except:` (bare) dangerous? Give one error it would hide that you'd rather see.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
class LikertError(ValueError):          # your own exception type
    pass
print(issubclass(LikertError, ValueError))  # -> True (so `except ValueError` still
catches it)

try:
    assert 1 == 2, "values differ"      # assert: cheap internal sanity check
except AssertionError as e:
    print(e)                            # -> values differ
```

In class — going deeper (second hour)

E1 — Which exception?

Without running, name the exception each raises: `int("3.5")` · `{"a": 1}["b"]` · `[1, 2][5]` · `1/0` · `open("nope.csv")` · `len(42)`

E2 — Your own exception

Define `class SurveyError(ValueError)` and raise it for out-of-range Likert values. Show that `except ValueError:` still catches it — that's subclassing at work.

D1 — Fix the except order

This always prints `"unexpected"` for bad cells — why, and what's the fix?

```
try:
    n = int(cell)
except Exception:
    print("unexpected")
except ValueError:
    n = None
```

D2 — `SurveyError` with evidence

Build `class SurveyError(ValueError)` that stores the offending value as `.value`. In your cleaning loop, catch the low-level `ValueError` from `int()` and re-raise as `SurveyError(cell, ...)` from `e`. Show the two-level traceback once.

D3 — Log, don't print

Convert your rejection log to `logging: logging.warning` per rejected cell, `logging.info` with the final counts. Configure the format to show the level name.

D4 — Parametrize

Rewrite your three homework tests as ONE `@pytest.mark.parametrize` test over `["3", True, None, 0, 9]`. Run `pytest -q`.

Homework (before Session 8)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — `ask_int(prompt, lo, hi)`

Session 3's validation loop, rebuilt the EAFP way: `try: n = int(raw) / except ValueError` instead of `.isdigit()` — so `"-5"` and `" 42 "` work too. Loop until valid; return the int.

H2 — Three more `pytest` cases

For `clean_likert`, add tests that: `clean_likert(True)` raises (a bool is not a rating!), `clean_likert("3")` raises (a str, even if numeric), `clean_likert(None)` raises. Run `pytest -q` until green.

H3 — Error triage

For each message, name the exception class and give the one-line fix:

- `invalid literal for int() with base 10: 'N/A'`
- `unsupported operand type(s) for +: 'int' and 'str'`
- `'score'` (raised by a dict lookup)
- `division by zero`
- `[Errno 2] No such file or directory: 'survy.csv'`

Solutions

In class

```
def safe_int(value):
    try:
        return int(value)
    except (ValueError, TypeError):
        return None

def clean_likert(n):
    if isinstance(n, bool) or not isinstance(n, int):
        raise ValueError(f"{n!r} not an int")
    if not 1 <= n <= 5:
        raise ValueError(f"{n} outside 1-5")
    return n

raw = ["5", "3", "N/A", "7", "", "1", "two", "4"]
clean, rejected = [], []
for r in raw:
    try:
        clean.append(clean_likert(safe_int(r)))
    except ValueError as e:
        rejected.append((r, str(e)))
print(clean)      # [5, 3, 1, 4]
print(rejected)   # [('N/A', ...), ('7', ...), ('', ...), ('two', ...)]
```

```
# test_clean.py
import pytest
from clean import clean_likert
def test_ok():    assert clean_likert(3) == 3
def test_bad():
    with pytest.raises(ValueError):
        clean_likert(9)
```

Task 4: a bare `except:` also catches `KeyboardInterrupt` (Ctrl+C) and `NameError` from your own typos — so a misspelled variable would be silently swallowed instead of showing you the bug. Always catch the specific exception you expect.

In class — going deeper

```
# E1
int("3.5")          # ValueError          (int() parses integer text only)
{"a": 1}["b"]       # KeyError
[1, 2][5]           # IndexError
1 / 0               # ZeroDivisionError
open("nope.csv")    # FileNotFoundError
len(42)             # TypeError           (ints have no length)

# E2
class SurveyError(ValueError):
    pass

def clean_likert(n):
    if not 1 <= n <= 5:
        raise SurveyError(f"{n} outside 1-5")
    return n

try:
    clean_likert(9)
except ValueError as e:      # the parent class catches the subclass
    print("caught:", e)
```



```

# D1 – except clauses are tried top-down; `Exception` is the parent of ValueError,
# so it matches FIRST and the specific handler is unreachable. Specific before broad:
try:
    n = int(cell)
except ValueError:
    n = None
except Exception as e:
    print("unexpected:", e)
    raise

# D2
class SurveyError(ValueError):
    def __init__(self, value, message):
        super().__init__(message)
        self.value = value

def clean_cell(cell):
    try:
        n = int(cell)
    except ValueError as e:
        raise SurveyError(cell, f"{cell!r} is not a rating") from e
    return n

# The traceback shows the ValueError as the cause, the SurveyError on top.

# D3
import logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
kept, rejected = [], []
for cell in ["5", "N/A", "3", ""]:
    try:
        kept.append(clean_cell(cell))
    except SurveyError as e:
        rejected.append(e.value)
        logging.warning("rejected %r", e.value)
logging.info("kept %d, rejected %d", len(kept), len(rejected))

# D4 – test_clean.py
import pytest
from clean import clean_likert

@pytest.mark.parametrize("bad", ["3", True, None, 0, 9])
def test_rejects(bad):
    with pytest.raises(ValueError):
        clean_likert(bad)

```

Homework

```
# H1
def ask_int(prompt, lo, hi):
    while True:
        raw = input(prompt)
        try:
            n = int(raw)          # EAFP: just try the conversion
        except ValueError:
            print("A whole number, please.")
            continue
        if lo <= n <= hi:
            return n
        print(f"Between {lo} and {hi}, please.")

# H2 – test_clean.py
import pytest
from clean import clean_likert

def test_bool_rejected():
    with pytest.raises(ValueError):
        clean_likert(True)

def test_str_rejected():
    with pytest.raises(ValueError):
        clean_likert("3")

def test_none_rejected():
    with pytest.raises(ValueError):
        clean_likert(None)
```

H3 — triage:

1. `ValueError` — clean/convert first, or wrap in `try/except ValueError`.
2. `TypeError` — convert one side: `str(n)` or `int(s)`.
3. `KeyError` — `row.get("score")`, or fix the header spelling.
4. `ZeroDivisionError` — guard the empty case before dividing.
5. `FileNotFoundError` — the filename is misspelled (`survey.csv`); check with `Path(...).exists()`.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
score = '3.0'    # a value from a CSV
int(score)
```

Result: `ValueError: invalid literal for int() with base 10: '3.0'`. `int()` parses INTEGER text only; '3.0' is not a valid int literal. Use `int(float(score))`.

2.

```
score = 2.5
round(score)
```

Result: `2`. Python uses banker's rounding (round half to EVEN): `round(2.5) == 2` but `round(3.5) == 4`. Watch this when summarising data.

3.

```
weight = 0.1 + 0.2 + 0.3    # three grade components
weight == 0.6
```

Result: `False`. Float error accumulates when you total components: `0.1+0.2+0.3` is `0.6000000000000001`. Compare sums with `math.isclose`, or round for display.

4.

```
enrolment = '1_000'
int(enrolment)
```

Result: `1000`. Python accepts underscores as digit separators, even inside `int()` text.

Session 8

Files, Libraries & Research Data

Opening files with `with`

```
with open("notes.txt") as f:
    text = f.read()
# file auto-closes here, even if the code crashes
```

`with` = a context manager: sets up and tears down the resource for you. Always prefer it to a bare `open()` / `close()`.

File modes (mind the trap)

Mode	Meaning
"r"	read (default)
"w"	write — truncates the file to empty first!
"a"	append
"r+"	read + write

⚠ Open the wrong file with `"w"` → its contents are gone.

```
with open("log.txt", "a") as f:      # append: adds to the end, never clobbers
    f.write("cleaned survey.csv\n")  # \n is YOUR job when writing
```

(`"rb"` / `"wb"` read/write **binary** — images, PDFs. Text mode is the default.)

Reading text

```
with open("notes.txt") as f:
    whole = f.read()      # one big string
    # or
    for line in f:         # line by line (memory-friendly)
        print(line.rstrip())
```

⚠ A file object is exhausted after one pass — re-open to read again.

CSV in, as dicts □

```
import csv
with open("students.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["score"])    # row is a dict keyed by header
```

`csv.DictReader` turns each row into a dict — your "list of dicts" dataset from Session 4. (`newline=""` avoids blank rows on Windows.)

You *could* split each line yourself with `.split(",")` — until a field contains a comma. That, plus quoting and headers, is why `csv` exists.

CSV out

```
with open("summary.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "score"])
    w.writeheader()
    w.writerow({"name": "Ana", "score": 91})
```

Libraries a researcher reaches for

```
import statistics
statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)

import random
random.choice(xs); random.randint(1, 6); random.shuffle(xs)

from datetime import date
date.today()

from pathlib import Path
Path("students.csv").exists()
```

`pip install <package>` for third-party libs.

The pandas teaser (your next course)

```
import pandas as pd
df = pd.read_csv("students.csv")
df["score"].describe()    # count, mean, std, min, quartiles, max
df.groupby("major")["score"].mean()
```

Everything you did by hand today — in three lines. We learned the fundamentals *underneath* it first.

Your turn

examples/session-08/practice.md (uses `survey.csv`):

1. Read `students.csv`; print class mean with `statistics.mean`.
 2. Compute per-item survey means; write `survey_summary.csv`.
-

Going deeper

Real data work

`pathlib`, the full tour

```
from pathlib import Path
data = Path("data")
f = data / "students.csv"          # / joins paths, any OS
f.exists(), f.stat().st_size       # is it there? how big?
list(data.glob("*.csv"))           # every CSV here
list(data.rglob("*.csv"))          # ...and in every subfolder
data.mkdir(exist_ok=True)
notes = f.with_suffix(".txt").read_text() # small files: one-liners
```

Encodings — the Excel gotcha

```
open("survey.csv", newline="", encoding="utf-8")          # say it explicitly
open("from_excel.csv", newline="", encoding="utf-8-sig") # eats Excel's BOM
```

- Mojibake (`Ã©` for `é`) = the bytes were read with the wrong encoding.
 - Excel often saves `cp1252` or UTF-8 *with a BOM* — `utf-8-sig` handles the latter.
-

`csv`, round 2

```
csv.reader(f)          # rows as LISTS (no/duplicate headers)
csv.DictReader(f)       # rows as DICTS (the default choice)
w.writerows(report)     # write many dicts at once
csv.DictReader(f, delimiter=";") # European Excel exports
```

JSON for nested data

```
import json
snapshot = {"course": "ED101",
            "items": {"q1": {"mean": 4.2, "n": 28}, "q2": {"mean": 3.8, "n": 27}}}
Path("snapshot.json").write_text(json.dumps(snapshot, indent=2))
back = json.loads(Path("snapshot.json").read_text())
```

CSV is a rectangle; JSON nests. `True/None` become `true/null` — JSON is its own language, `json.load` translates back.

The same JSON arrives from the **web**: `requests.get(url).json()` after a `pip install requests` — that's all an "API call" is. (A taste; offline files stay our home turf.)

`datetime` — dates are data too

```
from datetime import date, datetime
d = datetime.strptime("2026-07-06", "%Y-%m-%d")    # parse text -> datetime
d.strftime("%b %d")                               # format -> "Jul 06"
(date(2026, 7, 6) - date(2026, 1, 5)).days        # 182 – date math works
date.fromisoformat("2026-07-06")                  # the ISO shortcut
```

`strptime` = *parse* (string → date), `strftime` = *format* (date → string).

`random`, reproducibly

```
import random
random.seed(42)                                # same "random" every run – methods sections rejoice
random.choice(roster)                          # one
random.sample(roster, 3)                       # three, no repeats
random.shuffle(order)                          # in place – random assignment
```

Seeding makes stochastic analysis **reproducible** — rerunning your script re-draws the same sample.

Scripts that take arguments: `sys.argv`

```
# report.py – run as: python3 report.py survey.csv
import sys

if len(sys.argv) != 2:                          # argv[0] is the script's own name
    sys.exit("Usage: python3 report.py <file.csv>")
filename = sys.argv[1]
```

- `sys.argv` is a plain **list of strings** from the command line — `len()` it, slice it (`sys.argv[1:]` = just the real arguments).

- Guard first, then trust: a missing argument is an `IndexError` waiting to happen; `sys.exit("message")` stops the program and prints the usage line.
 - When flags multiply (`--top 5 --verbose`), graduate to the stdlib's `argparse` .
-

The pandas teaser, extended

```
import pandas as pd
df = pd.read_csv("students.csv")
df["score"].describe()           # count/mean/std/quartiles
df.groupby("major")["score"].agg(["mean", "count"])
df[df["score"] < 60]             # filter rows
df.to_csv("report.csv", index=False)
```

Five lines = today's whole session. That's the next course — and now you know what each line does underneath.

Your turn — round 2

`examples/session-08/practice.md` → **In class — going deeper:** a `pathlib` inventory, date math on enrollment dates, a seeded sample, and CSV → JSON.

Traps recap

- `"w"` silently overwrites — be sure of the filename.
- `csv` module → open with `newline=""` .
- Files exhaust after one read; re-open to re-read.
- Specify `encoding="utf-8"` for non-ASCII text.

Summary

You can load, summarize, and write real research data. **Next:** Session 9 — regular expressions & text cleaning.

Homework (before Session 9)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-08/practice.md` → Homework.

1. **Attendance report** — read a CSV, clean it, summarize it, write a report CSV — the whole pipeline.
2. **JSON round-trip** — save your summary with `json.dump` , load it back, verify equality.
3. **Your own data** — point the pipeline at any CSV from your research.

Practice & homework

In class

Files provided: `students.csv`, `survey.csv`.

Task 1 — Read & summarize students

Read `students.csv` with `csv.DictReader`. Remember the values are **strings** — convert `score` to `int`. Print the class mean and median with the `statistics` module.

Task 2 — Mean by major

Build `{major: mean_score}`. (Hint: `dict.setdefault(key, []).append(...)`.)

Task 3 — Clean & summarize the survey

`survey.csv` has "N/A" and blanks in numeric columns. For each `q*` item, compute the mean of the **valid** values only, and how many were valid. Write `survey_summary.csv` with columns `item, mean, n_valid`.

Task 4 — pandas teaser (optional)

If `pandas` is installed: `pd.read_csv("students.csv")["score"].describe()`. Compare the mean to your hand-computed one.

Trap check

What happens if you accidentally open `students.csv` with mode `"w"` before reading it?

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
import json
s = json.dumps({"n": 3, "ok": True})
print(s)                                # -> {"n": 3, "ok": true}    (Python True -> JSON true)
print(json.loads(s)["ok"])              # -> True                (and back to a Python
bool)
```

In class — going deeper (second hour)

E1 — One more column

Extend the survey summary: add a `pct_valid` column (share of rows with a usable value), formatted to one decimal.

E2 — pathlib mini-tour

With `from pathlib import Path`: does `students.csv` exist? How many bytes is it (`.stat().st_size`)? List every `.csv` in this folder (`.glob`).

D1 — pathlib inventory

List every `.csv` in this folder with its size in bytes, one aligned line each, using `Path(".").glob` and `.stat().st_size`.

D2 — Date math

Parse `"2026-01-05"` and `"2026-07-06"` with `strptime`, print the number of days between them, and print the first date as `Jan 05, 2026` with `strftime`.

D3 — A reproducible sample

Seed with `random.seed(42)`, then `random.sample` 3 names from the students in `students.csv`. Run it twice — same three? Why does a methods section care?

D4 — CSV → JSON

Read `students.csv` into a list of dicts with `int` scores, and write it to `students.json` with `indent=2`. Open the file: what happened to the quotes and numbers?

D5 — Take the filename from the command line

Turn Task 1 into `report.py`: read the CSV named on the command line (`python3 report.py students.csv`), print the class mean, and exit with a usage message when the argument is missing or extra. (`sys.argv`, `sys.exit`.)

Homework (before Session 9)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Attendance report (the whole pipeline)

Create `attendance.csv` yourself: a `name` column plus five `s1..s5` columns of 0/1, about six rows — and sneak in one dirty cell (`"?"`). Then: read it with `DictReader`, compute each student's attendance rate (skip dirty cells; remember that summing 0/1 ints just works), and write `attendance_report.csv` with `name,rate,n_valid` via `DictWriter`. Everything inside `with open(...)`.

H2 — JSON round-trip

`json.dump` your per-item survey means to `summary.json` (use `indent=2`), read it back with `json.load`, and verify `loaded == original`. Open the file in your editor — what happened to `True`?

H3 — Your own data

Point the pipeline at ANY CSV from your own work (export one from Excel/Sheets if needed): read it, count the rows, compute one mean, print a two-line report. Note the first dirty value you hit and how you handled it — into the bug log it goes.

Solutions

In class

See `demo.py` in this folder — it implements Tasks 1–3 exactly. Key lines:

```
scores = [int(s["score"]) for s in students]    # convert strings!
statistics.mean(scores)                        # 75.5

by_major = {}
for s in students:
    by_major.setdefault(s["major"], []).append(int(s["score"]))

def to_int(x):
    try: return int(x)
    except (ValueError, TypeError): return None    # handles N/A and ""
```

Trap: opening with `"w"` **truncates the file to empty immediately** — your data is gone before you ever read it. Use `"r"` (the default) to read.

In class — going deeper

```
# E1 — inside the loop over survey items
vals = [to_int(r[item]) for r in rows]
good = [v for v in vals if v is not None]
writer.writerow({"item": item,
                  "mean": round(statistics.mean(good), 2),
                  "n_valid": len(good),
                  "pct_valid": f"{100 * len(good) / len(vals):.1f}%"})

# E2
from pathlib import Path
p = Path("students.csv")
print(p.exists())                # True (in this folder)
print(p.stat().st_size)          # size in bytes
print(sorted(Path(".").glob("*.csv"))) # every CSV here
```

```

# D1
from pathlib import Path
for p in sorted(Path(".").glob("*.csv")):
    print(f"{p.name:<24}{p.stat().st_size:>8,} bytes")

# D2
from datetime import datetime
a = datetime.strptime("2026-01-05", "%Y-%m-%d")
b = datetime.strptime("2026-07-06", "%Y-%m-%d")
print((b - a).days)           # 182
print(a.strftime("%b %d, %Y")) # Jan 05, 2026

# D3
import csv, random
with open("students.csv", newline="", encoding="utf-8") as f:
    names = [r["name"] for r in csv.DictReader(f)]
random.seed(42)
print(random.sample(names, 3)) # same 3 every run – the seed pins the RNG,
                                # so your "random" sample is reproducible.

# D4
import json
with open("students.csv", newline="", encoding="utf-8") as f:
    rows = [{**r, "score": int(r["score"])} for r in csv.DictReader(f)]
Path("students.json").write_text(json.dumps(rows, indent=2))
# In the file: keys/strings get double quotes, scores sit bare (real JSON numbers).

```

```

# D5 – report.py
import csv, statistics, sys

if len(sys.argv) != 2:
    sys.exit("Usage: python3 report.py <file.csv>")

with open(sys.argv[1], newline="", encoding="utf-8") as f:
    scores = [int(r["score"]) for r in csv.DictReader(f)]
print(f"n={len(scores)} mean={statistics.mean(scores):.1f}")

```

Homework

```
# H1
import csv

with open("attendance.csv", newline="") as f:
    rows = list(csv.DictReader(f))

report = []
for r in rows:
    marks = []
    for key in list(r)[1:]:          # every column after "name"
        try:
            marks.append(int(r[key]))
        except ValueError:
            pass                    # skip the dirty cell – but count it as invalid
    rate = sum(marks) / len(marks) if marks else 0.0
    report.append({"name": r["name"], "rate": round(rate, 2), "n_valid": len(marks)})

with open("attendance_report.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "rate", "n_valid"])
    w.writeheader()
    w.writerows(report)

# H2
import json

original = {"q1": 4.2, "q2": 3.8, "all_valid": False}
with open("summary.json", "w") as f:
    json.dump(original, f, indent=2)
with open("summary.json") as f:
    loaded = json.load(f)
print(loaded == original)          # True – and in the file, Python's False is written as
                                   # JSON's false (lowercase): JSON is its own language.
```

H3 — a pattern, not a fixed answer: `rows = list(csv.DictReader(f))` inside a `with`, `len(rows)` for the count, one column through a `to_int / to_float` cleaner for the mean. The first dirty value and its exception belong in your bug log.

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
import csv, io
row = next(csv.DictReader(io.StringIO('name,score\nAna,91')))
row['score'] + 1
```

Result: `TypeError: can only concatenate str (not "int") to str`. Every CSV value arrives as a STRING — `'91' + 1` doesn't add, it crashes. Convert first: `int(row['score']) + 1`.

2.

```
import json
json.dumps({'passed': True})
```

Result: `'{"passed": true}'`. JSON is its own language: Python's `True` is written as lowercase `true` (and `None` becomes `null`). `json.load` translates them back.

3.

```
import io
f = io.StringIO('Ana\nBen\n') # stands in for an open file
first_pass = f.readlines()
f.readlines()
```

Result: `[]`. A file object is a cursor, not a container: after one pass it sits at the end and a second read returns nothing. Re-open the file (or `seek(0)`), or read it into a list once.

Session 9

Regular Expressions & Text Cleaning

Why a researcher cares

- Validate IDs, emails, dates before they pollute your data.
- Extract structured bits from free text (codes, names, numbers).
- Clean and normalize open-ended survey responses.
- A first pass at **qualitative coding** (find every response matching a pattern).

□ Like search-and-filter over a corpus — but it matches *form*, not meaning.

Always use raw strings

```
import re
re.search(r"\d+", "id 42")    # r"..." = raw string
```

Without `r"..."`, Python eats the backslashes (`\d` → error/garbage). **Rule:** every regex pattern is a raw string.

The survival tokens

Token	Matches
<code>.</code>	any char (except newline)
<code>\d \w \s</code>	digit / word-char / whitespace
<code>\D \W \S</code>	the negations
<code>+ * ?</code>	1+, 0+, 0-or-1
<code>{m} {m,n}</code>	exactly m / between m and n
<code>^ \$</code>	start / end of string
<code>[abc] [a-z] [^abc]</code>	any in set / in range / none in set
<code>(...)</code>	capture group
<code>a\b</code>	a or b

The `.` trap

```
re.search(r".", "a.b").group()    # 'a' - ANY char, not a dot!
re.search(r"\.", "a.b").group()   # '.' - escape it for a literal dot
```

Escape the specials when you mean them literally: `\. \^ \$ \* \+ \? \(\ \) \[ \] \{ \} \|`

The four functions you need

```
re.search(pattern, s)      # first match ANYWHERE -> match or None
re.fullmatch(pattern, s)   # the WHOLE string must match -> validation
re.findall(pattern, s)     # list of ALL matches
re.sub(pattern, repl, s)   # replace matches -> cleaning
```

`re.IGNORECASE` flag for case-insensitive: `re.search(p, s, re.IGNORECASE)`.

Validate (fullmatch anchors both ends)

```
def valid_university_email(addr):
    return re.fullmatch(r"\w+@\w+\.edu", addr) is not None

valid_university_email("ana@university.edu")    # True
valid_university_email("ana@gmail.com")         # False
valid_university_email("ana@x.edu")             # False
```

Extract with capture groups

```
m = re.search(r"([A-Z]{2})(\d{4})", "Course ED1234 meets Tue")
m.group(0)    # "ED1234"  whole match
m.group(1)    # "ED"      dept
m.group(2)    # "1234"    number
```

`m.groups()` hands back every capture at once: `('ED', '1234')`. `m` is `None` if nothing matched — check before `.group()`.

Clean & mine free text

```
re.sub(r"\s+", " ", messy).strip()      # collapse whitespace
re.findall(r"#(\w+)", "love #python #stats") # ['python', 'stats']

from collections import Counter
Counter(re.findall(r"#(\w+)", corpus))    # theme frequencies
```


Reformat with groups: `re.sub(r"^(.+),\s*(.+)$", r"\2 \1", "Curie, Marie")` → `"Marie Curie"`.

When NOT to use regex

```
"a,b,c".split(",")      # simple split – no regex needed
" hi ".strip()          # trim – no regex needed
text.replace("X", "Y")  # fixed substring – no regex needed
url.removeprefix("https://") # trim a known prefix – no regex needed (3.9+)
```

Regex shines for *variable* patterns. For fixed strings, plain methods read better.

Your turn

`examples/session-09/practice.md`:

1. Email validator.
 2. Extract dept+number.
 3. Collapse whitespace.
 2. Count hashtags across responses.
 5. Flip `"Last, First"`.
 6. One case to use `.split()` instead.
-

Going deeper

Power regex

Flags

```
re.findall(r"stress", corpus, re.IGNORECASE) # Stress, STRESS, stress
re.findall(r"^\d+", text, re.MULTILINE)      # ^ and $ match per LINE
re.search(p, s, re.IGNORECASE | re.MULTILINE) # combine with |
re.search(r"a.b", text, re.DOTALL)           # . crosses newlines too
```

`re.compile` — name your patterns

```
EMAIL = re.compile(r"\w+@\w+\.edu")
COURSE = re.compile(r"[A-Z]{2}\d{4}")

EMAIL.fullmatch(addr)      # same four functions, now methods
COURSE.findall(text)
```

Compile once, reuse everywhere — the pattern gets a *name* the reader can trust.

Readable patterns: `re.VERBOSE`

```
EMAIL = re.compile(r"""
    \w+          # the user part
    @
    \w+          # the domain
    \.edu        # a literal dot, then edu
""", re.VERBOSE)
```

Whitespace is ignored, comments allowed — a regex your future self can review.

Groups, round 2

```
m = re.search(r"(?P<dept>[A-Z]{2})(?P<num>\d{4})", "ED1234")
m.group("dept"), m.groupdict()      # names beat positions

r"(?:Prof|Dr)\.?\s+(\w+)"          # (?:...) groups WITHOUT capturing
```

Name the groups you extract; use `(?:...)` for grouping-only parentheses.

Greedy vs lazy

```
re.search(r"\[(.+)\]", "[ED101][ED102]").group(1)  # 'ED101][ED102' ⓘ
re.search(r"\[(.+?)\]", "[ED101][ED102]").group(1) # 'ED101'
```

`+` and `*` grab as much as possible; `+?` / `*?` as little as possible. Bracketed/quoted things almost always want **lazy**.

`re.sub` with a function: the anonymizer

```
ids = {}
def anonymize(m):
    name = m.group(0)
    ids.setdefault(name, f"P{len(ids) + 1:03d}")
    return ids[name]

re.sub(r"\b(?:Ana|Ben|Cara)\b", anonymize, transcript)
# "P001 said ... P002 replied ... P001 agreed"
```

When the replacement must be *computed*, pass a function — every match goes through it. (A closure over `ids` — Session 5 pays off.)

Regex meets files

```
text = Path("responses.txt").read_text(encoding="utf-8")
emails = sorted(set(EMAIL.findall(text)))
```

Session 8 + Session 9 in two lines: read the file, mine the pattern, dedupe with a set.

Your turn — round 2

`examples/session-09/practice.md` → **In class — going deeper:** a commented `VERBOSE` email pattern, the anonymizer, and a two-format date harvest.

Traps recap

- `.` matches **any** char — use `\.` for a literal dot.
- Forgetting `r"..."` breaks your backslashes.
- `re.search` returns `None` on no match — guard before `.group()`.
- Don't use regex where a string method is clearer.

Summary

You can validate, extract, and clean real-world text. **Next:** Session 10 — modules, OOP & the Pythonic toolkit.

Homework (before Session 10)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-09/practice.md` → Homework.

1. **Pattern drill** — student IDs, US phone numbers, ISO dates: one `fullmatch` pattern each.
2. **Messy-name cleanup** — normalize a scraped name column with `re.sub`.
3. **Domain harvest** — extract every email domain from a block of text.

Practice & homework

In class

Always use raw strings `r"..."`. Predict each result before running.

Task 1 — Validate

Write `valid_university_email(addr)` returning `True` only for `something@something.edu`. Test: `"ana@university.edu"`, `"ana@gmail.com"`, `"a@b.eduevil.com"`.

Task 2 — Extract with groups

From `"Course ED1234 meets Tue"`, pull the department (`ED`) and number (`1234`) using one regex with two capture groups.

Task 3 — Clean

Collapse all runs of whitespace in `" too much\t space "` to single spaces and trim.

Task 4 — Mine free text

From a list of open-ended responses, count how often each `#hashtag` appears (use `re.findall(r"#(\w+)", text)` and `collections.Counter`).

Task 5 — Reformat

Turn `"Curie, Marie"` into `"Marie Curie"` with a single regex + groups.

Task 6 — Judgment

Give one task where a plain string method (`.split()`, `.strip()`, `.replace()`) is the better, clearer choice than a regex.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
import re
m = re.search(r"(?P<year>\d{4})", "class of 2026")
print(m.group("year"), m.groupdict()) # -> 2026 {'year': '2026'} (named groups)
print(re.split(r"\s*,\s*", "a, b ,c")) # -> ['a', 'b', 'c'] (split on commas + spaces)
```

In class — going deeper (second hour)

E1 — Named groups

Redo the dept+number extraction with `(?P<dept>...)` / `(?P<num>...)` and print `m.groupdict()`.

E2 — regex101 field trip

Paste your email pattern into regex101.com (flavor: **Python**). Read the left-panel explanation token by token — does it say what you *meant*?

D1 — A commented pattern

Rewrite your email validator with `re.compile(..., re.VERBOSE)`, one commented line per token. It must behave identically.

D2 — The anonymizer

Replace every occurrence of the names Ana/Ben/Cara in a transcript with stable IDs `P001`, `P002`, ... (same name → same ID) using `re.sub` with a **function** replacement.

D3 — Two-format date harvest

From `"submitted 2026-07-06, revised 07/08/2026, due 2026-09-01"`, extract **all** dates in either `YYYY-MM-DD` or `MM/DD/YYYY` form with one `findall` and an alternation.

D4 — Case-insensitive keyword count

Count mentions of "stress" in a paragraph regardless of case, with a flag — not by lowercasing the text.

Homework (before Session 10)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Pattern drill

One `re.fullmatch` pattern each; test against two valid and two invalid strings:

1. Student ID: two uppercase letters + six digits (`AB123456`)
2. US phone: `555-867-5309` (digits and dashes only)
3. ISO date: `2026-07-06` (just the shape — don't validate month ranges)

H2 — Messy-name cleanup

Normalize `[" smith, ana", "LEE,BEN", "Garcia , Cara "]` to `"Ana Smith"`, `"Ben Lee"`, `"Cara Garcia"`: regex-split on the comma (with optional spaces around it), then `.strip()` + `.title()`, and flip the order.

H3 — Domain harvest

From a paragraph containing several email addresses, extract the **unique** domains (e.g. `{"university.edu", "gmail.com"}`) with one `findall` + a `set`.

Solutions

In class

See `demo.py` in this folder — it implements all six. Key lines:

```

re.fullmatch(r"\w+@\w+\.edu", addr) is not None      # 1 (fullmatch anchors both ends)
m = re.search(r"([A-Z]{2})(\d{4})", s); m.group(1), m.group(2)  # 2
re.sub(r"\s+", " ", messy).strip()                  # 3
from collections import Counter; Counter(re.findall(r"#(\w+)", text))  # 4
m = re.search(r"^(.+),\s*(.+)$", s); f"{m.group(2)} {m.group(1)}"    # 5

```

Task 6: splitting `"a,b,c"` on commas is just `"a,b,c".split(",")` — no regex needed. Reach for regex only when the pattern is genuinely variable (digits, optional parts, anchors).

Trap reminder: `.` matches **any** character — use `\.` for a literal dot, and never forget the `r"..."` prefix or your backslashes become Python escape sequences.

In class — going deeper

```

# E1
import re
m = re.search(r"(?P<dept>[A-Z]{2})(?P<num>\d{4})", "Course ED1234 meets Tue")
print(m.group("dept"), m.group("num"))    # ED 1234
print(m.groupdict())                     # {'dept': 'ED', 'num': '1234'}

```

E2 — the point is the habit: regex101's explainer catches "`.` matches any character" mistakes before your data does.

```

import re

# D1
EMAIL = re.compile(r"""
    \w+          # the user part
    @
    \w+          # the domain
    \.edu        # a literal dot, then edu
""", re.VERBOSE)
print(EMAIL.fullmatch("ana@university.edu") is not None)    # True

# D2
ids = {}
def anonymize(m):
    name = m.group(0)
    ids.setdefault(name, f"P{len(ids) + 1:03d}")
    return ids[name]

text = "Ana said X. Ben said Y. Ana agreed."
print(re.sub(r"\b(?:Ana|Ben|Cara)\b", anonymize, text))
# P001 said X. P002 said Y. P001 agreed.

# D3
s = "submitted 2026-07-06, revised 07/08/2026, due 2026-09-01"
print(re.findall(r"\d{4}-\d{2}-\d{2}|\d{2}/\d{2}/\d{4}", s))
# ['2026-07-06', '07/08/2026', '2026-09-01']

# D4
para = "Stress was high. Some stress is normal. STRESS!"
print(len(re.findall(r"stress", para, re.IGNORECASE)))    # 3

```

Homework

```
# H1
import re

sid    = r"[A-Z]{2}\d{6}"          # AB123456 ✓   CD000001 ✓   ab123456 ✗   AB12345 ✗
phone  = r"\d{3}-\d{3}-\d{4}"      # 555-867-5309 ✓   555-8675309 ✗
date   = r"\d{4}-\d{2}-\d{2}"      # 2026-07-06 ✓   2026-7-6 ✗

for s in ["AB123456", "ab123456"]:
    print(s, re.fullmatch(sid, s) is not None)

# H2
def clean_name(raw):
    last, first = re.split(r"\s*,\s*", raw.strip(), maxsplit=1)
    return f"{first.strip().title()} {last.strip().title()}"

for raw in [" smith, ana", "LEE,BEN", "Garcia , Cara "]:
    print(clean_name(raw))          # Ana Smith · Ben Lee · Cara Garcia

# H3
text = "Write ana@university.edu or ben@gmail.com; cc cara@university.edu today."
print(set(re.findall(r"\w+@([\w.]+\.\w+)", text)))
# {'university.edu', 'gmail.com'} - the set removes the duplicate
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
import re
m = re.match(r'\[(.+)\]', '[ED101][ED102]')
m.group(1)
```

Result: `'ED101][ED102'`. `+` is greedy — it grabs as much as it can. Use `+?` for the shortest match: `r'\[(.+?)\]'`.

2.

```
import re
bool(re.match(r'\d+', 'cohort2026'))
```

Result: `False`. `re.match` anchors at the START of the string. Use `re.search` to find a match anywhere.

3.

```
import re
re.fullmatch(r'grades.csv', 'gradesXcsv') is not None
```

Result: `True`. `.` matches ANY character, so `gradesXcsv` fits the pattern. Escape it — `r'grades\.csv'` — when you mean a literal dot.

Session 10

Modules, OOP & the Pythonic Toolkit

Modules: split your code into files

```
# grades.py
def letter_grade(score): ...
def class_average(scores): ...

# analysis.py
from grades import letter_grade, class_average
```

A `.py` file is a **module**. `import` reuses its functions elsewhere → no copy-paste.

The `__main__` guard

```
# grades.py
if __name__ == "__main__":
    print(letter_grade(85)) # runs ONLY when you execute grades.py directly
```

On `import grades`, `__name__` is `"grades"`, so the block is skipped. One file can be both a runnable script *and* an importable library.

OOP: model a domain entity

```
class Student:
    def __init__(self, name, gpa): # constructor
        self.name = name
        self.gpa = gpa
    def __str__(self): # how it prints
        return f"{self.name} ({self.gpa})"

ana = Student("Ana", 3.9)
print(ana) # Ana (3.9)
```

□ A class is an *operational definition*: the attributes + behaviors that "count" as a Student. `self` = "this particular student."

@property: validate on assignment

```
class Student:
    ...
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa must be 0-4")
        self._gpa = value
```

`ana.gpa = 5.0` now raises — the object defends its own integrity.

Inheritance

```
class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)    # reuse parent setup
        self.advisor = advisor
    def __str__(self):
        return super().__str__() + f" - {self.advisor}"
```

`GradStudent` is a `Student` plus extra. `super()` calls the parent.

The Pythonic toolkit (recap tour)

```
[s.name for s in roster if s.gpa >= 2.0]    # comprehension (from S4)
list(map(lambda s: s.name.upper(), roster)) # map: apply to all
list(filter(lambda s: s.gpa < 2.0, roster)) # filter: keep matches
for i, s in enumerate(roster): ...          # index + item
for a, b in zip(names, scores): ...         # parallel
```

Generators & walrus

```
def gpas(students):
    for s in students:
        yield s.gpa    # one value at a time – low memory on big data

g = gpas(roster)
list(g)    # ?
list(g)    # ? ← run it twice – what changes? (Traps below)

if (n := len(roster)) > 30:    # walrus := : assign + test in one step
    print(f"{n} students")
```

Your turn

examples/session-10/practice.md :

1. Import from `grades.py` . 2. Build the validating `Student` class.
 2. Add `GradStudent` with `super()` . 4. Comprehension + `map` + `filter` + a generator.
-

Going deeper

Designing with objects

`__repr__` and friends

```
class Student:
    def __repr__(self):
        # for developers (REPL, lists, logs)
        return f"Student({self.name!r}, {self.gpa})"
    def __eq__(self, other):
        return (self.name, self.gpa) == (other.name, other.gpa)
    def __lt__(self, other):
        # "less than" – sorted() now just works
        return self.gpa < other.gpa

sorted(roster)                                # no key= needed anymore
```

Dunders make your objects behave like built-ins: printable, comparable, sortable — even **addable**: define `__add__(self, other)` and `total = quiz + final` just works. That's all "operator overloading" is.

`@dataclass`, round 2

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    gpa: float = 0.0
    courses: list = field(default_factory=list)    # NOT courses: list = []
```

`default_factory=list` is the mutable-default rule (Session 5!) in class form — a fresh list per instance. `@dataclass(frozen=True)` gives you an immutable record.

@classmethod : alternate constructors

```
@dataclass
class Student:
    name: str
    score: int

    @classmethod
    def from_row(cls, row):
        # a CSV row (Session 8) -> an object
        return cls(row["name"], int(row["score"]))

roster = [Student.from_row(r) for r in csv.DictReader(f)]
```

`cls` is the class itself. Pattern: **file format at the edge, objects inside**. (`@staticmethod` is the third flavor: a plain helper that lives in the class's namespace — no `self`, no `cls`.)

Composition over inheritance

```
class Cohort:
    # a Cohort HAS Students (composition ✓)
    def __init__(self, name):
        self.students = []

class GradStudent(Student):
    # a GradStudent IS a Student (inheritance ✓)
    ...
```

Inherit only for a true *is-a*. When in doubt, hold objects inside other objects — it stays flexible and testable.

Class vs instance attributes — the rule

```
class Course:
    school = "Ed School"
    # class attr: SHARED by all – constants only
    def __init__(self):
        self.students = []
    # instance attr: each object its own
```

Mutable data goes in `__init__`, always. (You met the shared-list trap — this is the law it teaches.)

Generator pipelines

```
def to_ints(rows):
    for r in rows:
        try:
            yield int(r)
        except ValueError:
            pass

def in_range(vals, lo=1, hi=5):
    yield from (v for v in vals if lo <= v <= hi)

clean = list(in_range(to_ints(raw)))    # nothing runs until consumed
```

Each stage yields to the next — **lazy**, constant-memory, works on a million-row file.

From script to project

```
survey_tool/
├─ clean.py          # validation functions
├─ stats.py          # summaries
├─ report.py         # main: python3 report.py
└─ tests/test_clean.py
```

One module per concern; `report.py` imports the rest; tests alongside. Inside `report.py`, the convention:

```
def main():
    # the program's real work, in one place
    ...

if __name__ == "__main__":
    main()
```

You already know every ingredient — this is just arranging them.

Your turn — round 2

`examples/session-10/practice.md` → **In class — going deeper**: sortable `Student` s, a dataclass with `default_factory`, `from_row`, and a two-stage generator pipeline.

Traps recap

- `self` is just "this instance" — not magic.
- A generator iterates **once**, then it's empty.
- The `__main__` guard keeps imported modules from running their demo code.

- Don't reach for a class when a function or dict will do.

Summary

You can structure code into modules and classes and write idiomatic Python. **Next (optional):** Session 11 — the capstone: put it all together.

Homework (before the capstone)

Outside class — it doesn't count toward class time. Full specs + solutions: `examples/session-10/practice.md` → Homework.

1. `Student.courses` — a courses list and a *computed* `gpa` — data plus the rules that guard it.
2. A **cohort class** — holds `Student` s; `mean_gpa()`, `at_risk()`.
3. **Pythonic rewrite** — three given loops → comprehension, generator, `map`.

Practice & homework

In class

Files in this folder: `grades.py` (a module you import), `demo.py` (worked example).

Task 1 — Use a module

From a new file, `from grades import letter_grade, class_average` and call both. Why does the `if __name__ == "__main__":` block in `grades.py` NOT run when you import it?

Task 2 — A class with a validating property

Build `Student(name, gpa)` with:

- `__str__` → `"Ana: 3.9 (Good)"`,
- `standing()` → `"Good"` if `gpa ≥ 2.0` else `"Probation"`,
- a `@property` setter for `gpa` that raises `ValueError` outside 0–4. Prove the setter rejects `5.0`.

Task 3 — Inheritance

Add `GradStudent(Student)` that also stores an `advisor` and uses `super().__init__(...)`. Override `__str__` to append the advisor.

Task 4 — The Pythonic toolkit

Given a roster of `Student` s:

1. names in good standing (list comprehension),
2. uppercase names (`map`),

3. at-risk students (`filter`),
4. mean gpa via a **generator** that `yield`s each gpa — then show the generator is empty on a second pass.

Bonus — Pythonic idiom drill

Cover the `# ->` answers, predict each line, then run.

```
from dataclasses import dataclass

@dataclass                                # auto __init__, __repr__, __eq__
class Point:
    x: int
    y: int
print(Point(1, 2), Point(1, 2) == Point(1, 2))    # -> Point(x=1, y=2) True

def head(v):
    match v:                                # structural pattern matching (3.10+)
        case [first, *_]: return first
        case _: return None
print(head([9, 8]), head(5))                  # -> 9 None
```

In class — going deeper (second hour)

E1 — The guard, observed

Add `print("running as", __name__)` to the top of `grades.py`. Run the file directly, then `import grades` from the REPL — what prints each time, and why?

E2 — `@dataclass` Student

Rebuild the plain (non-validating) `Student` as a `@dataclass`. What do you get for free? Check `repr` and `==` on two equal students.

D1 — Sortable students

Give `Student` a `__repr__`, `__eq__`, and `__lt__` (by gpa) so that `sorted(roster)` works with `no key=`. Prove it.

D2 — `dataclass` done right

Write `@dataclass class Course: name: str; roster: list = field(default_factory=list)`. Why would `roster: list = []` be wrong — and which Session 5 bug is this the class-shaped version of?

D3 — `from_row`

Add `Student.from_row(cls, row)` building a `Student` from a CSV `DictReader` row (convert types!). Why is a `@classmethod` the right home for this?

D4 — A two-stage pipeline

Write generators `to_ints(cells)` (skip unparseable) and `in_range(vals, lo=1, hi=5)`; chain them over `["5", "3", "N/A", "7", "1"]`. When does any work actually happen?

Homework (before the capstone)

~30–45 minutes, outside class — it doesn't count toward class time. Try everything before peeking at the solutions.

H1 — Courses and a computed GPA

Extend `Student`: a `courses` list of `(name, grade_points)` tuples, an `add_course(name, points)` method that validates 0–4, and make `gpa` a **computed property** (mean of the course points; `None` with no courses). No stored `gpa` that can go stale.

H2 — A cohort class

`Cohort(name)` holds `Student` `s`: `add(student)`, `mean_gpa()` (skip students with no `gpa`), `at_risk(threshold=2.0)` returning names, and `__str__` → `"Fall-26 (3 students)"`. Careful where the student list lives — remember the class-variable trap.

H3 — Pythonic rewrite

Rewrite each as a single comprehension / generator / `map` line:

```
# 1 -> list comprehension
out = []
for s in roster:
    if s.gpa is not None and s.gpa >= 3.5:
        out.append(s.name)

# 2 -> generator inside sum() (no list built)
total = 0
for s in roster:
    total += len(s.courses)

# 3 -> map (or a comprehension – say which you prefer, and why)
labels = []
for s in roster:
    labels.append(str(s))
```

Solutions

In class

See `demo.py` — it implements Tasks 2–4. Key points:


```

@property
def gpa(self): return self._gpa
@gpa.setter
def gpa(self, v):
    if not 0 <= v <= 4: raise ValueError(...)
    self._gpa = v

class GradStudent(Student):
    def __init__(self, name, gpa, advisor):
        super().__init__(name, gpa)
        self.advisor = advisor

[s.name for s in roster if s.gpa >= 2.0]          # comprehension
list(map(lambda s: s.name.upper(), roster))      # map
[s.name for s in filter(lambda s: s.gpa < 2.0, roster)] # filter
def gpas(rs):
    for s in rs: yield s.gpa                    # generator (exhausts after one pass)

```

Task 1: the `__name__` guard is only `"__main__"` when the file is **run directly**; on `import` its `__name__` is `"grades"`, so the demo block is skipped — that's how a file can be both a runnable script and an importable module.

In class — going deeper

```

# E1
# python3 grades.py    -> "running as __main__" (the file IS the program)
# >>> import grades    -> "running as grades"   (so the __main__ block is skipped)

# E2
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    gpa: float

# Free: __init__, a readable __repr__, and field-by-field __eq__:
print(Student("Ana", 3.9) == Student("Ana", 3.9)) # True

```

```

# D1
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa
    def __repr__(self):
        return f"Student({self.name!r}, {self.gpa})"
    def __eq__(self, other):
        return (self.name, self.gpa) == (other.name, other.gpa)
    def __lt__(self, other):
        return self.gpa < other.gpa

roster = [Student("Ana", 3.9), Student("Ben", 1.8), Student("Cara", 3.2)]
print(sorted(roster))    # Ben, Cara, Ana – __lt__ told sorted() how

# D2
from dataclasses import dataclass, field

@dataclass
class Course:
    name: str
    roster: list = field(default_factory=list)
# `roster: list = []` would share ONE list across every Course – the mutable-default
# bug from Session 5, in class form. default_factory makes a fresh list per instance.

# D3
@dataclass
class Student2:
    name: str
    score: int

    @classmethod
    def from_row(cls, row):
        return cls(row["name"], int(row["score"]))
# It constructs an instance of the class itself, so it belongs to the CLASS (cls),
# not to any one object – the "alternate constructor" pattern.

# D4
def to_ints(cells):
    for c in cells:
        try:
            yield int(c)
        except ValueError:
            pass

def in_range(vals, lo=1, hi=5):
    yield from (v for v in vals if lo <= v <= hi)

pipeline = in_range(to_ints(["5", "3", "N/A", "7", "1"]))
print(list(pipeline))    # [5, 3, 1] – nothing ran until list() pulled values through

```

Homework

```
# H1
class Student:
    def __init__(self, name):
        self.name = name
        self.courses = []                # (course, grade_points) tuples

    def add_course(self, course, points):
        if not 0 <= points <= 4:
            raise ValueError(f"{points} outside 0-4")
        self.courses.append((course, points))

    @property
    def gpa(self):                        # computed – can never go stale
        if not self.courses:
            return None
        return sum(p for _, p in self.courses) / len(self.courses)

    def __str__(self):
        return f"{self.name} (GPA {self.gpa})"

# H2
class Cohort:
    def __init__(self, name):
        self.name = name
        self.students = []              # per-instance – NOT a class variable!

    def add(self, student):
        self.students.append(student)

    def mean_gpa(self):
        gpas = [s.gpa for s in self.students if s.gpa is not None]
        return sum(gpas) / len(gpas) if gpas else None

    def at_risk(self, threshold=2.0):
        return [s.name for s in self.students
                if s.gpa is not None and s.gpa < threshold]

    def __str__(self):
        return f"{self.name} ({len(self.students)} students)"

# H3
names = [s.name for s in roster if s.gpa is not None and s.gpa >= 3.5]
total = sum(len(s.courses) for s in roster)      # generator – no list in memory
labels = list(map(str, roster))                  # or [str(s) for s in roster] – either is fine;
                                                # most Pythonistas find the comprehension clearer
```

Traps — predict, then check

Cover the result, predict each one, then check yourself.

1.

```
gpas = (s for s in [3.9, 1.8, 3.2])
first_pass = list(gpas)
list(gpas)
```

Result: `[]`. A generator is one-shot: after the first full pass it is exhausted. Rebuild it, or store a list if you need it twice.

2.

```
from dataclasses import dataclass

@dataclass
class Grade:
    course: str
    score: int

g1 = Grade('ED101', 91)
g2 = Grade('ED101', 91)
g1 == g2
```

Result: `True`. `@dataclass` writes an `__eq__` that compares fields, so equal-valued grades are `==` (even though `g1 is g2` is `False`).

3.

```
class Course:
    students = []
    def enroll(self, name):
        self.students.append(name)

art = Course()
math = Course()
art.enroll('Ana')
math.students
```

Result: `['Ana']`. `students` is a CLASS variable shared by every instance. Give each its own in `__init__`: `self.students = []`.

Setup & Tools

Everything you need to run Python on your own machine — plus the tools (and AI habits) that make learning this material faster.

Fastest start: no install at all

Every example on this site runs **in your browser** — just press **Run**. Nothing to install, great for the first sessions. When you want to work on *your own files*, install Python locally with the steps below.

Run the course as Jupyter notebooks

Every session is also a **Jupyter notebook** — the same examples and practice as a runnable, cell-by-cell document. Three ways to use them; pick whatever fits:

- **In your browser, no install** — open the live JupyterLite workspace: <https://yangnei.github.io/learn-python/jupyter/lab/index.html>. It's real Jupyter running entirely in your browser; your edits are saved locally in the browser, nothing is uploaded. Each session page also has a **Run in browser** button that opens that session's notebook directly.
- **Google Colab** — each session page has an **Open in Colab** button (free, runs in Google's cloud, needs a Google account). Handy when you want to save to Drive or do heavier data work.
- **Local Jupyter** — once Python is installed (below): `pip install jupyterlab`, then run `jupyter lab` and open the `.ipynb` you saved with a session page's **Download .ipynb** button.

How to drive a notebook: click a cell and press **Shift + Enter** to run it and move to the next. Predict the output before you run — the surprise is the lesson — then edit a cell and re-run to experiment.

Autocomplete & libraries: press **Tab** to complete a name and **Shift + Tab** to pop up a function's help. To add a package, run `%pip install <name>` *in a cell* (e.g. `%pip install pandas`). In the browser (JupyterLite) only Pyodide-compatible packages work — the big ones like `pandas`, `numpy`, and `matplotlib` do — and the install lasts for the session, so re-run that cell after a kernel restart. In local Jupyter or Colab, `%pip install` is permanent for that environment.

Install Python (pick your OS)

Windows

- Official installer: <https://www.python.org/downloads/windows/> — on the first screen, **check "Add python.exe to PATH"**, then *Install Now*.
- (Alternative) Microsoft Store → search **Python 3.12**.
- Check it worked — open **PowerShell** and run: `python --version`

- Official guide: <https://docs.python.org/3/using/windows.html>

macOS

- Official installer: <https://www.python.org/downloads/macos/>
- (Alternative) **Homebrew**: `brew install python`
- Check it worked — open **Terminal** and run: `python3 --version`
- Official guide: <https://docs.python.org/3/using/mac.html>

Linux

- Usually already installed. If not: `sudo apt install python3 python3-pip` (Debian/Ubuntu) or `sudo dnf install python3` (Fedora).
- Check it worked: `python3 --version`
- Official guide: <https://docs.python.org/3/using/unix.html>

An editor: install **VS Code** (<https://code.visualstudio.com/>) and its **Python extension** (by Microsoft). Step-by-step: <https://code.visualstudio.com/docs/python/python-tutorial>. Prefer something lighter? **Thonny** (<https://thonny.org/>) is a beginner IDE with a built-in variable viewer and debugger.

Run a script: save your code as `name.py`, then in a terminal run `python name.py` (Windows) or `python3 name.py` (macOS/Linux).

Install a package (e.g. for the Session 8 pandas teaser): `pip install pandas` (or `pip3`).

Visualize what your code does — Python Tutor

<https://pythontutor.com/> — paste code and **step through it line by line**, watching variables, the call stack, and which names point at which objects. It's the single best tool for the things this course keeps warning you about:

- **aliasing** — see `b = a` make both names point at the *same* list (Session 4),
- the **mutable-default-argument** bug (Session 5),
- the **recursion call stack** building up and unwinding (Session 6).

Whenever you think "*wait, why did that happen?*" — drop the snippet into Python Tutor.

Use AI to learn (without it doing the work for you)

An AI assistant (Claude, etc.) is a huge accelerator *if you ask it to explain rather than to write*. Type every solution yourself. Prompts that fit this course:

- "Summarize this doc page for a beginner, just the 5 things I need: `\<paste text or URL>`" — turn a dense reference page into something usable.
- "Explain this error and its most likely cause, and point me to the line: `\<paste the traceback>`"

- **"Predict what this prints, then explain why: \<code>"** — then run it and compare. That predict-then-run habit *is* the method this course is built on.
- **"Review my function for the traps in this course — identity vs value, aliasing, mutable defaults, off-by-one — but don't rewrite it for me."**
- **"Give me 3 harder practice tasks like this one, with the solutions hidden."**

Rule of thumb: **ask it to explain, not to do**. If it hands you code you can't read line by line, ask it to slow down and walk you through it.

Other handy tools

- **regex101.com** (<https://regex101.com/>) — build and *explain* regular expressions interactively, with a live match view (Session 9). Set the flavor to **Python**.
- **Official docs** — the [Python Tutorial](#) and the [Library Reference](#). Bookmark both.
- **Google Colab** (<https://colab.research.google.com/>) — run Python notebooks in the browser, no install; a natural next step once you start doing real data work.
- This site's **Traps & Gotchas** sheet — keep it open the whole course.

Python Traps & Gotchas — The Master Cheat Sheet

The quirks that bite beginners (and plenty of experts). Every behavior below was run and verified on CPython 3.11+. Read the **Surprise** column, cover the **Why/Fix**, and predict. Keep this open for the whole course.

1 — Equality `==` vs Identity `is` (Session 2)

Expression	Result	
<code>a = [1,2]; b = [1,2]; a == b</code>	<code>True</code>	same value
<code>a = [1,2]; b = [1,2]; a is b</code>	<code>False</code>	different objects in memory
<code>a = [1,2]; b = a; a is b</code>	<code>True</code>	<code>b</code> is the <i>same</i> object (an alias)

- `==` asks "same value?" — this is what you almost always want.
- `is` asks "same object?" — use it **only** for `None`, `True`, `False`: `python if x is None: # ✓`
`correct if x == None: # ⚠ works, but not idiomatic – use `is``
- **Identity wrinkle (do NOT rely on this):** CPython caches small integers (–5 to 256) and interns some strings, so `a = 256; b = 256; a is b` is `True` but at `257` it may be `False`. This is an implementation detail. **For value, always use `==`.**

□ *Bridge:* two students with the same GPA are `==`; the same student is `is`.

2 — Booleans ARE integers (Session 2)

```
True == 1          # True    (bool is a subclass of int)
False == 0         # True
5 + True          # 6        yes, you can do arithmetic on bools
sum([True, False, True]) # 2 ← counts the Trues
isinstance(True, int)    # True
type(True) is int      # False (its type is bool, a subtype)
```

- **Fix / use it well:** `sum(conditions)` is the Pythonic way to *count* how many are true.
- **Trap:** `True is 1` is `False` (different objects) even though `True == 1`. Compare with `==`.

□ *Bridge:* this is dummy coding (1/0) built into the language — summing flags counts cases.

3 — Float precision (Session 2)

```
0.1 + 0.2          # 0.30000000000000004
0.1 + 0.2 == 0.3    # False 🤖
```

- **Why:** decimals are stored in binary; most can't be represented exactly. Not a Python bug — every language does this.
- **Fix:** `python import math math.isclose(0.1 + 0.2, 0.3) # True` ← compare floats with a tolerance `round(0.1 + 0.2, 2) == 0.3 # True` ← or round for display/compare from decimal `import Decimal # exact decimal arithmetic when you truly need it (money, grades) Decimal("0.1") + Decimal("0.2") # Decimal('0.3')`
- **Rule:** never test computed floats with `==`. Use `math.isclose` (or `round`).

□ *Bridge:* you already never test two measured scores for exact equality — same instinct, new cause (binary storage, not measurement error).

4 — Comparing across types (Session 2)

```
5 == "5"          # False (different types → not equal, but NO error)
5 == 5.0           # True (int vs float compare by numeric value)
5 > "5"            # ❌ TypeError: '>' not supported between 'int' and 'str'
```

- **Equality** (`== / !=`) across unrelated types → just `False`, never crashes.
- **Ordering** (`<, >, <=, >=`) across incompatible types → `TypeError`.
- **Fix:** convert first. `int("5") == 5 → True`. Guard with `isinstance` before ordering.

□ *Bridge:* the computer can equate "are these the same?" across types, but can't *rank* text against numbers — it has no shared scale.

5 — Sequences compare element-by-element (Session 2 / 4)

```
[1, 2] == [1, 2]    # True
[1, 2] == (1, 2)     # False ← list vs tuple: different types, never equal
(1, 2) < (1, 3)       # True ← compares position by position (lexicographic)
"apple" < "banana"    # True ← strings compare char by char (alphabetical-ish)
[1, 2] < [1, 2, 3]    # True ← prefix is "less than"
```

- **Trap:** a `list` and a `tuple` with identical contents are **never** `==` — type matters.
- Lists/tuples/strings compare left-to-right; the first differing element decides.

6 — Truthiness (what counts as False) (Session 2 / 3)

```
# These are all "falsy":
bool(0) bool(0.0) bool("") bool([]) bool({}) bool(set()) bool(None) # all False
bool("0") # True! ← a non-empty string is truthy, even "0"
bool([0]) # True ← a non-empty list is truthy, even if it holds a 0
```

- **Fix / idiom:** check emptiness directly — `if scores: not if len(scores) > 0;` ; `if name: not if name != "":` .
- **Trap:** `"0"` (string) and `"False"` (string) are **truthy**. Convert user input before testing.

7 — `and` / `or` return an operand, not a bool (Session 3)

```
5 and 0 # 0 (and → first falsy, or last value)
0 or "hi" # "hi" (or → first truthy, or last value)
"" or "N/A" # "N/A" ← handy default-value idiom
```

- **Use it well:** `name = user_input or "Anonymous"` supplies a default.
- **Trap:** don't write `if x == True` . Just `if x:` . And `and` / `or` short-circuit — the right side may never run, so don't hide side effects there.

8 — Mutable default arguments (Session 5) — the famous one

```
def add_student(name, roster=[]): # ✗ DANGER
    roster.append(name)
    return roster

add_student("Ana") # ['Ana']
add_student("Ben") # ['Ana', 'Ben'] ☹️ the default list PERSISTS across calls
```

- **Why:** the default `[]` is created **once**, when the function is *defined*, and reused every call.
- **Fix:** `python def add_student(name, roster=None): # ✓ if roster is None: roster = []`
`roster.append(name) return roster`
- **Rule:** never use a mutable default (`[]` , `{}` , `set()`). Use `None` and create inside.

9 — Aliasing: variables are labels, not boxes (Session 4)

```
a = [1, 2, 3]
b = a          # b is the SAME list, not a copy
a.append(4)
b              # [1, 2, 3, 4] 🤖 b changed too

b = a.copy()   # ✓ now b is an independent (shallow) copy
import copy
b = copy.deepcopy(a) # ✓ independent even for nested lists/dicts
```

- **Trap:** `grid = [[0] * 3] * 3` makes **three references to one row** — editing `grid[0][0]` changes all rows. Use `[[0]*3 for _ in range(3)]`.
- **Rule:** assignment never copies. `=` binds another name to the same object.

□ *Bridge:* a variable is a sticky label on an object, not a container holding a value.

10 — `type()` VS `isinstance()` (Session 2)

```
isinstance(x, int)          # ✓ Pythonic; respects inheritance
isinstance(x, (int, float)) # ✓ "is x any kind of number?"
type(x) is int              # exact type only; rarely what you want
type(x) == int              # works but use `is` for type identity
```

- **Trap:** `isinstance(True, int)` is `True` (bool is a subtype of int). If you must exclude bools, check `type(x) is int` OR `isinstance(x, int)` and not `isinstance(x, bool)`.

11 — Integer vs float division (Session 1 / 2)

```
7 / 2      # 3.5    true division ALWAYS returns a float
7 // 2     # 3      floor division (rounds toward -∞)
-7 // 2    # -4     ← floors, doesn't truncate toward zero!
7 % 2      # 1      modulo (remainder) — use % 2 to test even/odd
7 / 0      # ❌ ZeroDivisionError
```

- **Trap:** `/` gives a float even when the result is whole (`4 / 2` is `2.0`). Use `//` for integer results.

12 — Strings are immutable; some methods *return*, don't mutate (Session 1 / 9)

```
s = " Hello "
s.strip()    # "Hello" ← returns a NEW string
s           # " Hello " ← s is UNCHANGED
s = s.strip() # ✓ reassign to keep the result
```

- **Trap:** `s.strip()` / `s.replace(...)` / `s.upper()` do nothing to `s` unless you reassign.
- Contrast: list methods like `.append()` / `.sort()` mutate **in place** and return `None`: `python nums = [3, 1, 2]` `nums = nums.sort()` # ✗ now `nums` is `None`! `.sort()` returns `None` `nums.sort()` # ✓ sorts in place; use `sorted(nums)` to get a new list

13 — `range` excludes the stop; off-by-one (Session 3)

```
list(range(1, 5))      # [1, 2, 3, 4] ← 5 is NOT included
list(range(5))         # [0, 1, 2, 3, 4]
list(range(0, 10, 2))  # [0, 2, 4, 6, 8]
```

- **Rule:** `range(a, b)` is the half-open interval `[a, b)`.

14 — Modifying a list while iterating it (Session 3)

```
xs = [1, 2, 3, 4]
for x in xs:
    if x % 2 == 0:
        xs.remove(x)    # ✗ skips elements / unpredictable
# ✓ iterate a copy, or build a new list:
xs = [x for x in xs if x % 2 != 0]
```

15 — Files: `"w"` overwrites; cursor exhausts (Session 8)

```
open("data.csv", "w")    # ! TRUNCATES the file to empty immediately
open("data.csv", "a")    # append
open("data.csv", "r")    # read (default)

with open("data.csv") as f:
    rows = f.readlines()  # read once
    again = f.readlines() # [] – cursor is at end of file now
```

- Always prefer `with open(...)` as `f`: — it closes the file even if the code crashes.
- With the `csv` module on Windows, open with `newline=""` to avoid blank rows.

16 — Regex: `.` matches *anything*; use raw strings (Session 9)

```
import re
re.search(r"\d+", "id 42")    # use r"..." so \d isn't a Python escape
re.search(".", "a.b")         # the "." matches "a", not just the dot!
re.search(r"\.", "a.b")       # use \. for a literal dot
```

- **Rule:** always write patterns as raw strings `r"..."`. Escape `.` `^` `$` `*` `+` `?` `(` `)` `[` `]` `{` `}` `|` `\`.

Quick "predict-then-run" self-test (cover the answers)

```
print(True + True)           # 2
print(3 == 3.0)              # True
print(0.1 + 0.2 == 0.3)      # False
print(5 == "5")              # False
print([1,2] == (1,2))        # False
print(bool("0"))             # True
print(7 // 2, -7 // 2)        # 3 -4
x = [1]; y = x; x.append(2); print(y)  # [1, 2]
```

If you can explain *why* for all eight, you've got the fundamentals most beginners miss.

Python Quick Reference — Syntax You'll Forget

The "how do I write that again?" sheet. Grouped by session.

I/O & types (S1)

```
name = input("Name: ")      # ALWAYS returns a str
age = int(input("Age: "))    # convert immediately
print("Hi", name, age)       # space-separated
print(f"Hi {name}, age {age}") # f-string (preferred)
print(f"{score:.2f}")        # 2 decimals
print(f"{count:,}")          # thousands separator: 1,234
print("no newline", end="")  # suppress the trailing \n
int("42"), float("3.14"), str(42), bool(0) # casts
type(x)                      # what type is x?
```

f-string formatting mini-table

Spec	Example	Output
<code>:.2f</code>	<code>f"{3.14159:.2f}"</code>	<code>3.14</code>
<code>:,</code>	<code>f"{1234567:,}"</code>	<code>1,234,567</code>
<code>:>8</code>	<code>f"{'hi':>8}"</code>	<code>hi</code> (right-align)
<code>:<8</code>	<code>f"{'hi':<8}"</code>	<code>hi</code> (left-align)
<code>^8</code>	<code>f"{'hi':^8}"</code>	<code>hi</code> (center)
<code>:.1%</code>	<code>f"{0.873:.1%}"</code>	<code>87.3%</code>

Conditionals (S3)

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "F"

90 <= score <= 100          # chained comparison
grade = "pass" if score >= 60 else "fail"  # ternary

match command:               # Python 3.10+
    case "start": ...
    case "stop" | "halt": ... # multiple values
    case _: ...               # default
```

Loops (S3)

```
for i in range(5): ...        # 0..4
for x in items: ...           # each element
for i, x in enumerate(items): ... # index + element
for a, b in zip(names, scores): ... # walk two lists together
while condition: ...
while True:                   # validation loop
    x = input("...")
    if valid(x):
        break
# break / continue control flow
```

Sequences & slicing (S4)

```
xs = [1, 2, 3]; xs.append(4); xs[0]; xs[-1]  # last
xs[1:3]    # [2, 3] (start inclusive, stop exclusive)
xs[:2]     # first two
xs[::-1]   # reversed
t = (1, 2) # tuple (immutable)
d = {"name": "Ana", "gpa": 3.9} # dict
d["name"]; d.get("missing", 0) # access; safe access w/ default
d.keys(); d.values(); d.items()
s = {1, 2, 2, 3}                # set -> {1, 2, 3} (unique)
```

Comprehensions (S4/S10)

```
[x*2 for x in xs]              # list
[x for x in xs if x > 0]        # with filter
{name: 0 for name in names}    # dict
{x % 3 for x in xs}            # set
(x*x for x in xs)              # generator (lazy)
```

Sorting (S4)

```
sorted(xs)                                # new sorted list
sorted(xs, reverse=True)
sorted(students, key=lambda s: s["gpa"])    # by a field
sorted(students, key=lambda s: s["gpa"], reverse=True)
xs.sort()                                  # in place (returns None!)
```

Functions (S5)

```
def avg(nums: list[float]) -> float:
    """Return the mean of nums."""    # docstring
    return sum(nums) / len(nums)

def greet(name, greeting="Hello"): ... # default arg (NOT mutable!)
def total(*args): ...                  # any number of positionals -> tuple
def config(**kwargs): ...              # any number of keywords -> dict
func(*my_list)                         # unpack list into args
func(**my_dict)                       # unpack dict into kwargs
```

Exceptions (S7)

```
try:
    n = int(value)
except ValueError:
    n = None
except (KeyError, IndexError) as e:
    print(e)
else:
    print("ok")          # ran only if no exception
finally:
    print("always")      # cleanup, always runs

raise ValueError("score must be 1-5")
assert n > 0, "n must be positive"
```


Files & CSV (S8)

```
with open("f.txt") as f:          # read, auto-closes
    text = f.read()
with open("f.txt", "w") as f:     # write (OVERWRITES)
    f.write("line\n")

import csv
with open("data.csv", newline="") as f:
    for row in csv.DictReader(f):  # row is a dict keyed by header
        print(row["name"])

with open("out.csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=["name", "gpa"])
    w.writeheader()
    w.writerow({"name": "Ana", "gpa": 3.9})
```

Handy stdlib (S8)

```
import statistics; statistics.mean(xs); statistics.median(xs); statistics.stdev(xs)
import random; random.choice(xs); random.randint(1, 6); random.shuffle(xs)
from datetime import date; date.today(); date(2026, 6, 26)
from pathlib import Path; Path("data.csv").exists()
import json; json.dumps(obj, indent=2); json.loads(text)
```

Regex (S9)

```
import re
re.search(r"pattern", text)      # first match anywhere (or None)
re.fullmatch(r"\d{5}", zip)     # whole string must match
re.sub(r"\s+", " ", text)       # collapse whitespace
m = re.search(r"(\w+)@(\w+)", s)
m.group(1)                       # first capture group
```

Token	Means
.	any char (except newline)
\d \w \s	digit / word-char / whitespace
+ * ?	1+, 0+, 0-or-1
{m,n}	between m and n
^ \$	start / end
[...] [^...]	set / not-in-set
(...)	capture group
a b	a or b

Classes (S10)

```
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self._gpa = gpa
    def __str__(self):
        return f"{self.name} ({self._gpa})"
    @property
    def gpa(self):
        return self._gpa
    @gpa.setter
    def gpa(self, value):
        if not 0 <= value <= 4:
            raise ValueError("gpa out of range")
        self._gpa = value

s = Student("Ana", 3.9)
print(s)           # uses __str__
```

The program skeleton (*every script*)

```
def main():
    ...

def helper():
    ...

if __name__ == "__main__":
    main()
```

Glossary — Plain-Language Definitions

For a learner who knows research but not code. Each term: what it is, in one breath.

- **Argument** — a value you hand to a function when you call it. `int("5")` — "5" is the argument.
- **Parameter** — the named slot in a function definition that receives an argument. `def f(x):` — `x` is the parameter.
- **Variable** — a *name* (label) pointing at an object. Not a box; a sticky note.
- **Object** — any value Python stores: a number, string, list, function, even a type. "Everything is an object."
- **Type** — the kind of an object: `int`, `float`, `str`, `bool`, `list`, `dict`, `set`, `tuple`, `NoneType`.
- **Dynamic typing** — variables aren't locked to one type; the *object* has a type, the *name* doesn't.
- **int / float** — whole number / decimal number. `/` always makes a float; `//` floors to int.
- **str** — text. Immutable (can't change in place). Written in quotes.
- **bool** — `True` / `False`. Technically a kind of `int` (`True == 1`).
- **None** — "no value." Returned by functions that don't `return` anything. Test with `is None`.
- **f-string** — `f"...{expr}..."` — a string with `{}` placeholders filled by Python.
- **Equality (`==`)** — "same value?" The comparison you usually want.
- **Identity (`is`)** — "literally the same object in memory?" Use only for `None` / `True` / `False`.
- **Truthy / Falsy** — any value used in an `if` counts as true unless it's `0`, `""`, `[]`, `{}`, `set()`, or `None`.
- **Mutable / Immutable** — can it be changed in place? `list` / `dict` / `set` yes; `int` / `str` / `tuple` no.
- **Aliasing** — two names pointing at the *same* mutable object; changing one shows in the other.
- **Slicing** — taking a sub-sequence: `xs[start:stop:step]`, stop excluded.
- **Comprehension** — a one-line way to build a list/dict/set from a loop: `[x*2 for x in xs]`.
- **Function** — a reusable named block of code; takes inputs, optionally `return`s an output.
- **return vs print** — `return` hands a value back to the *caller*; `print` just shows text to the screen.
- **Scope** — where a name is visible. Python looks Local → Enclosing → Global → Built-in (LEGB).
- ***args / **kwargs** — collect "any number of" positional args (as a tuple) / keyword args (as a dict).
- **Type hint** — optional annotation of expected types (`def f(x: int) -> str:`). Documentation, **not** enforced at runtime.
- **Exception** — an error raised at runtime (e.g., `ValueError`). Handle with `try / except`.
- **raise** — deliberately throw an exception. **EAFP** — "easier to ask forgiveness than permission": try it, catch failure.
- **assert** — a developer sanity-check that errors if false. Not for validating user input (can be turned off).
- **Module** — a `.py` file you can `import`. **Library / package** — a bundle of modules (install via `pip`).
- **Standard library** — modules that ship with Python (`csv`, `statistics`, `random`, `re`, `json`, `datetime`, `pathlib`).
- **Context manager (`with`)** — a block that sets up and tears down a resource automatically (e.g., closes a file).
- **CSV** — comma-separated values; a plain-text table. `csv.DictReader` turns each row into a dict.

- **Class / Object (OOP)** — a blueprint (`class`) and the things built from it (instances). Bundles data + behavior.
- `self` — inside a class, "this particular instance."
- `__init__` — the setup method run when you create an instance. **Dunder** = "double underscore" method.
- `@property` — makes a method look like an attribute; lets you validate on get/set.
- **Generator / `yield`** — produces values one at a time, lazily; saves memory on big data. Can be iterated once.
- `lambda` — a tiny anonymous function, often used as a `key=` for sorting.
- **Regex (regular expression)** — a pattern language for matching/extracting text (`re` module).
- **Traceback** — the error report Python prints when something fails. **Read the last line first.**
- **REPL** — the interactive `>>>` prompt where you type one line at a time.

Per-Session Quizzes (with answer keys)

Each quiz is 5–6 questions (the last one or two from the Going-deeper hour), ~8 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Running Python, Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`? And what is `int(3.9)`?
3. What does `f"{0.873:.1%}"` produce?
4. In a traceback, which line do you read first — and what two things does it tell you?
5. What are `17 // 5` and `17 % 5` — and name one practical use of `%`.

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`; `int(3.9)` is `3` (truncates, doesn't round). 3. `"87.3%"`. 4. The **last** line: the exception's name (what kind of problem) and the message (the offending value or detail). 5. `3` and `2`; `%` gives the remainder — even/odd tests, rotating IDs into k groups, splitting minutes into h + min.

Session 2 — The Dynamic-Typing Traps

1. `a = [1,2]; b = [1,2]` — is `a == b`? Is `a is b`? Why?
2. What is `True + True`? Why?
3. Is `0.1 + 0.2 == 0.3` True or False? How *should* you compare them?
4. What does `5 == "5"` give? What about `5 > "5"`?
5. Which of these are falsy: `"0"`, `""`, `[0]`, `[]`, `None`, `0.0`?
6. Why does `Decimal("0.1") + Decimal("0.2") == Decimal("0.3")` hold when the float version doesn't — and why build Decimals from strings?

Answers: 1. `==` True (same value), `is` False (different objects in memory).

2. `2` — `bool` is a subclass of `int` (`True` acts as `1`). 3. False (binary float rounding); use `math.isclose(0.1 + 0.2, 0.3)` or `round`. 4. False (no error); `5 > "5"` raises `TypeError` — Python can't *order* int vs str. 5. `""`, `[]`, `None`, `0.0` are falsy; `"0"` and `[0]` are truthy (non-empty). 6. `Decimal` stores base-10 digits exactly, so there's no binary rounding; built from a float (`Decimal(0.1)`) it would inherit the float's error.
-

Session 3 — Control Flow: Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.

2. What is `5` and `0`? What is `0` or `"hi"`? Why aren't they `True` / `False`?
3. What does `list(range(1, 5))` produce?
4. What goes wrong when you `remove` items from a list while looping over it — and what's the fix?
5. When does the `else` clause on a `for` loop run?
6. With `score = 95`, three stacked `if score >= 60 / 80 / 90:` prints run — what happens, and what one keyword fixes it?

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and` / `or` return an *operand*, not a bool. 3. `[1, 2, 3, 4]` (5 excluded — off-by-one). 4. Removal shifts the remaining indices, so elements get skipped; build a new list instead (`[x for x in xs if keep(x)]`) or iterate over a copy. 5. Only when the loop finishes **without** `break` — the "searched everything, found nothing" branch. 6. All three labels print (each `if` is an independent question); chaining with `elif` makes one ladder where only the first hit runs.

Session 4 — Data Structures

1. Which are mutable: list, tuple, dict, set?
2. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
3. (Practical) Write a one-line dict comprehension mapping each name in `names` to `0`.
4. `xs.sort()` vs `sorted(xs)` — what does each return?
5. You need "have I seen this ID before?" over thousands of IDs — which structure, and why?
6. What does `Counter(['a', 'b', 'a']).most_common(1)` return?

Answers: 1. list, dict, set are mutable; tuple is not. 2. `[1, 2, 3]` — `b` is an alias for the *same* list (assignment doesn't copy). 3. `{n: 0 for n in names}`.

4. `.sort()` sorts in place and returns `None`; `sorted()` returns a *new* sorted list.
 5. A `set` — membership tests are fast and duplicates are impossible by construction. 6. `[('a', 2)]` — a list of (value, count) pairs, biggest first.
-

Session 5 — Functions, Scope & Reusability

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function with no `return` statement return? Are type hints enforced at runtime?
4. `count = 0` at top level, then inside a function `count = count + 1` — what error, and why?
5. In one sentence: what does a decorator do, and what is `@d` above a `def f` sugar for?

Answers: 1. `return` hands a value to the caller; `print` only displays it.

2. The default list is created once, at `def` time, and persists across calls; use `items=None` then create inside.
 3. `None` ; and no — hints are documentation (`mypy` checks them optionally).
 4. `UnboundLocalError` — the assignment makes `count` local to the function, so the right-hand side reads a local that doesn't exist yet.
 5. It wraps a function with extra behavior; `@d` is exactly `f = d(f)` .
-

Session 6 — Recursion & Recursive Thinking

1. What two parts must every recursive function have?
2. What error comes from a base case that's never reached, and why doesn't Python just keep going?
3. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?
4. The recursive case computes `n * f(n - 1)` but has no `return` in front — what does a call produce?
5. Naive recursive `fib(35)` takes seconds but `@functools.cache` makes it instant — what does the cache change?

Answers: 1. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, *toward* the base case). 2. `RecursionError` — Python has no tail-call optimization, so each pending call keeps a stack frame until the limit (~1000).

3. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and reaches every level.
 4. `None` — the value is computed and thrown away; the function falls off the end.
 5. Each distinct input is computed once and remembered, so the exponential tree of repeated subcalls collapses into ~35 lookups.
-

Session 7 — Exceptions & Defensive Code

1. Which exception does `int("N/A")` raise? Wrap `int(value)` so it returns `None` on failure.
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert` ?
4. Name the two validation styles — and give the EAFP version of converting `value` to an int.
5. Why must `except ValueError:` come before `except Exception:` ?

Answers: 1. `ValueError` ; `try: return int(value) / except (ValueError, TypeError): return None` .

2. It catches *everything* (even Ctrl+C and your own typos) and can hide bugs.

3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (it can be disabled with `python -O`). 4. LBYL ("look before you leap": `if value.isdigit():`) vs EAFP ("easier to ask forgiveness"): `try: n = int(value) / except ValueError: ...`. 5. Handlers are tried top-down and `Exception` matches a `ValueError` too — placed first it would swallow everything, leaving the specific handler unreachable.
-

Session 8 — Files, Libraries & Research Data

1. What does opening a file in `"w"` mode do to existing contents? Why prefer `with open(...)`?
2. After `csv.DictReader`, what type is each row?
3. CSV values read from a file are what type — and what must you do with numbers?
4. Your second loop over an open file object sees nothing — why, and what's the fix?
5. What do `strptime` and `strftime` each do — and why seed `random` in an analysis script?
6. Your script runs as `python3 report.py survey.csv` — what is `sys.argv`, and what sits at index 0?

Answers: 1. Truncates it to empty *immediately*; `with` auto-closes the file even if the code crashes. 2. A `dict` keyed by the header row. 3. Strings; convert with `int()` / `float()`. 4. The file cursor is exhausted after one pass — re-open the file (or read it into a list first). 5. `strptime` parses text → datetime; `strftime` formats datetime → text; seeding makes the "random" sample reproducible for your methods section. 6. A plain list of strings, `['report.py', 'survey.csv']`; index 0 is always the script's own name, so real arguments start at 1 — check `len(sys.argv)` and `sys.exit("usage...")` before trusting them.

Session 9 — Regular Expressions & Text Cleaning

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?
3. What does `re.search` return when there's no match, and what must you do before `.group()`?
4. `re.search` vs `re.fullmatch` — which one is for validation, and why?
5. What does `re.IGNORECASE` change, and what is `re.VERBOSE` for?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. It returns `None`; check `if m:` before `m.group()` or you'll hit an `AttributeError`. 4. `fullmatch` — it anchors both ends, so the *whole* string must fit the pattern; `search` would accept a valid-looking fragment inside garbage. 5. `IGNORECASE` matches regardless of case; `VERBOSE` lets a pattern span lines with comments so a colleague can actually review it.

Session 10 — Modules, OOP & the Pythonic Toolkit

1. In a class, what is `self` ?
2. What happens if you iterate a generator twice?
3. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?
4. `class Course: students = []` — what goes wrong when two `Course` objects enroll students, and what's the fix?
5. In a dataclass, why write `courses: list = field(default_factory=list)` instead of `= []` ?

Answers: 1. The current instance ("this particular object"). 2. The second pass is empty — a generator is exhausted after one iteration. 3. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped. 4. `students` is a *class* variable shared by every instance, so both courses see one combined list; give each its own in `__init__`: `self.students = []`. 5. A bare `[]` default would be created once and shared by every instance — the Session 5 mutable-default bug in class form; `default_factory` builds a fresh list per instance.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the next session (or the capstone).
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 2 items:** revisit the type traps before continuing — they're load-bearing for everything after.